



(12) **United States Patent**
Manevich et al.

(10) **Patent No.: US 12,401,507 B2**
(45) **Date of Patent: Aug. 26, 2025**

(54) **FUTURE ASSET RECLAMATION VIA
BLOCKCHAIN**

(71) Applicant: **International Business Machines
Corporation**, Armonk, NY (US)

(72) Inventors: **Yacov Manevich**, Haifa (IL); **Artem
Barger**, Haifa (IL); **Nitin Gaur**,
Roundrock, TX (US); **Petr Novotny**,
Mount Kisco, NY (US)

(73) Assignee: **International Business Machines
Corporation**, Armonk, NY (US)

(*) Notice: Subject to any disclaimer, the term of this
patent is extended or adjusted under 35
U.S.C. 154(b) by 674 days.

(21) Appl. No.: **17/565,058**

(22) Filed: **Dec. 29, 2021**

(65) **Prior Publication Data**

US 2023/0208638 A1 Jun. 29, 2023

(51) **Int. Cl.**
H04L 9/30 (2006.01)
H04L 9/06 (2006.01)

(52) **U.S. Cl.**
CPC **H04L 9/30** (2013.01); **H04L 9/0618**
(2013.01)

(58) **Field of Classification Search**
CPC H04L 9/3247; H04L 9/0618; H04L 9/30;
H04L 9/50
See application file for complete search history.

(56) **References Cited**

U.S. PATENT DOCUMENTS

10,812,274 B2 10/2020 Back et al.
12,175,458 B2* 12/2024 Bergner G06Q 20/3829

12,212,696 B2* 1/2025 Chai H04L 63/0442
2014/0254796 A1* 9/2014 Li H04L 9/3263
380/246
2017/0155628 A1* 6/2017 Rohloff H04L 63/02
2017/0345019 A1* 11/2017 Radocchia H04W 12/10
2018/0089729 A1* 3/2018 Metnick G06Q 30/018
2018/0183587 A1* 6/2018 Won H04L 63/166
2018/0191503 A1 7/2018 Alwar et al.
2019/0251554 A1* 8/2019 Ma H04L 9/0869
2019/0260592 A1 8/2019 Nguyen et al.
2019/0394175 A1 12/2019 Zhang et al.

(Continued)

FOREIGN PATENT DOCUMENTS

CN 109768987 A 5/2019

OTHER PUBLICATIONS

Belchoir et al. "A survey on blockchain interoperability: Past,
present, and future trends." arXiv preprint arXiv:2005.14282 (2020).,
Mar. 2020.

(Continued)

Primary Examiner — Mohammad A Siddiqi

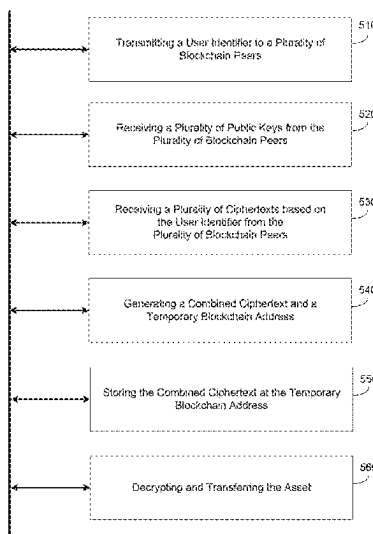
(74) *Attorney, Agent, or Firm* — CHIP LAW GROUP

(57) **ABSTRACT**

An example operation may include one or more of trans-
mitting a user identifier to a plurality of blockchain peers of
a blockchain network, receiving a plurality of public keys
from the plurality of blockchain peers, receiving a plurality
of ciphertexts from the plurality of blockchain peers gener-
ated based on the user identifier and a corresponding plu-
rality of private keys, generating a combined ciphertext that
includes the plurality of ciphertexts and generating a tem-
porary blockchain address of the blockchain network based
on the plurality of public keys, and storing the combined
ciphertext and a transfer of an asset via the temporary
blockchain address of the blockchain network.

20 Claims, 22 Drawing Sheets

500



(56)

References Cited

U.S. PATENT DOCUMENTS

2020/0127828 A1* 4/2020 Liu H04L 9/30
2021/0089676 A1 3/2021 Ford et al.
2022/0029831 A1* 1/2022 Baek G16Y 10/75
2022/0150073 A1* 5/2022 Androulaki G16H 10/60
2022/0255739 A1* 8/2022 Xu H04L 9/0618
2022/0407725 A1* 12/2022 Chai H04L 9/50
2022/0407729 A1* 12/2022 Zhang H04L 63/0435
2023/0205850 A9* 6/2023 Bernardi G06F 21/64
705/59
2024/0195618 A1* 6/2024 Paul H04L 9/14

OTHER PUBLICATIONS

Harish et al. "Log-flock: A blockchain-enabled platform for digital asset valuation and risk assessment in E-commerce logistics financing." Computers & Industrial Engineering 151 (2021): 107001.

Kolb et al. "Core concepts, challenges, and future directions in blockchain: a centralized tutorial." ACM Computing Surveys (CSUR) 53.1 (2020): 1-39, Feb. 2020.

Neven, "Efficient sequential aggregate signed data." IEEE transactions on information theory 57.3 (2011): 1803-1815.

Pillai et al, "The Burn-to-Claim cross-blockchain asset transfer protocol." 2020 25th International Conference on Engineering of Complex Computer Systems (ICECCS). IEEE, Oct. 2020.

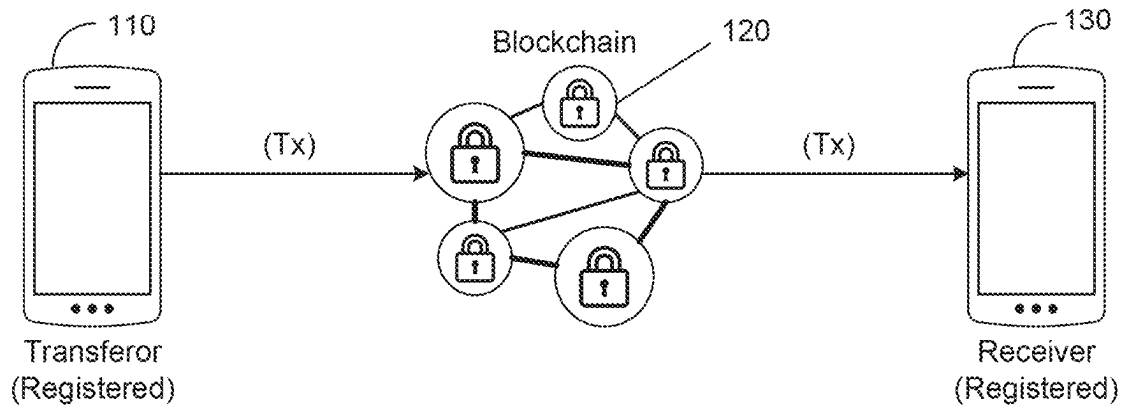
Boshmaf et al. BlockTag: Design and Applications of a Tagging System for Blockchain Analysis, HAL Open Science, Aug. 2, 2022, 16 pages.

Nazimunnisa et al. "BSSPD: A Blockchain-Based Security Sharing Scheme for Personal Data with Fine-Grained Access Control", Neuroquantology, Nov. 2022, pp. 5530-5541, vol. 20, Issue 19.

* cited by examiner

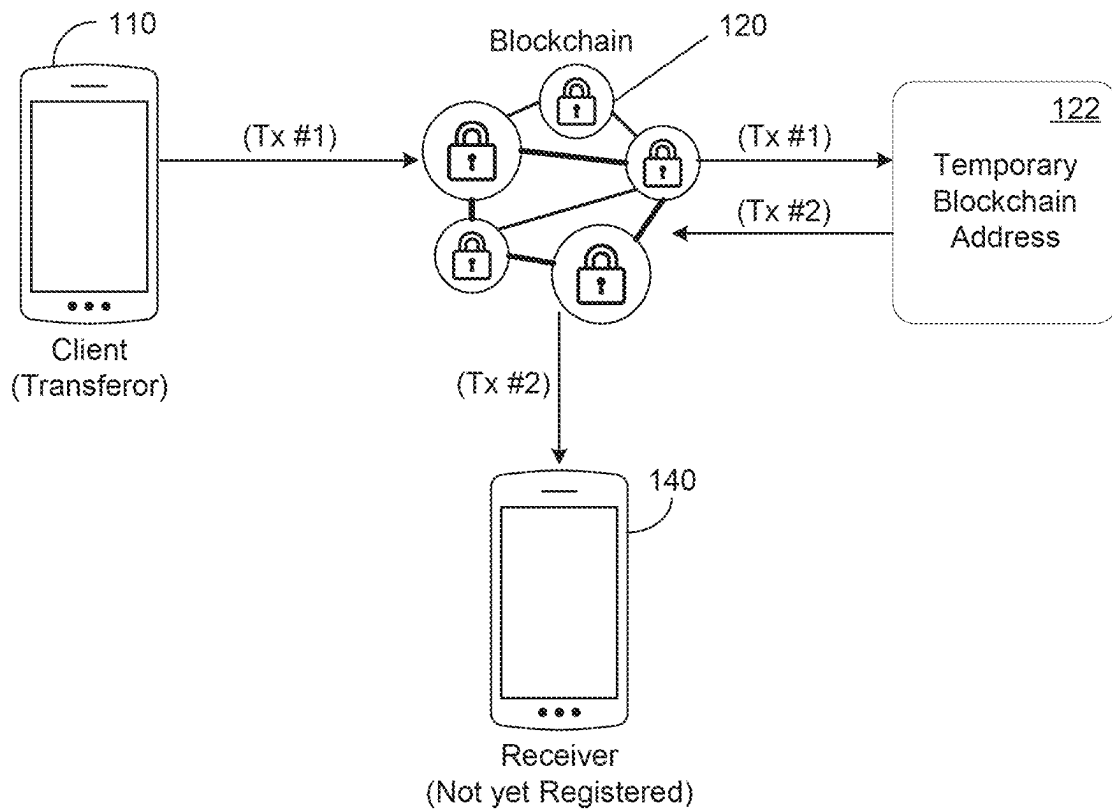
100A

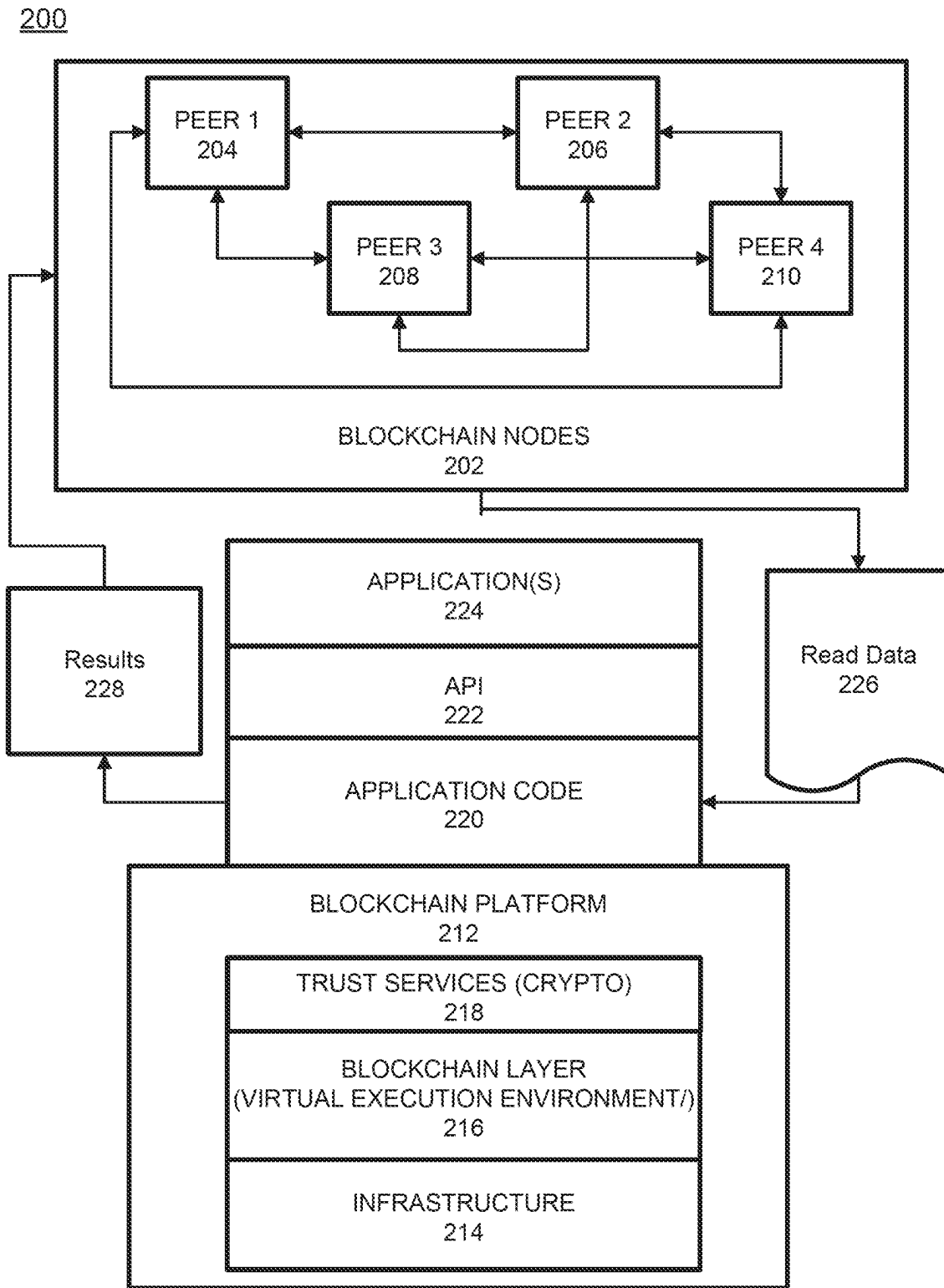
FIG. 1A



100B

FIG. 1B





250

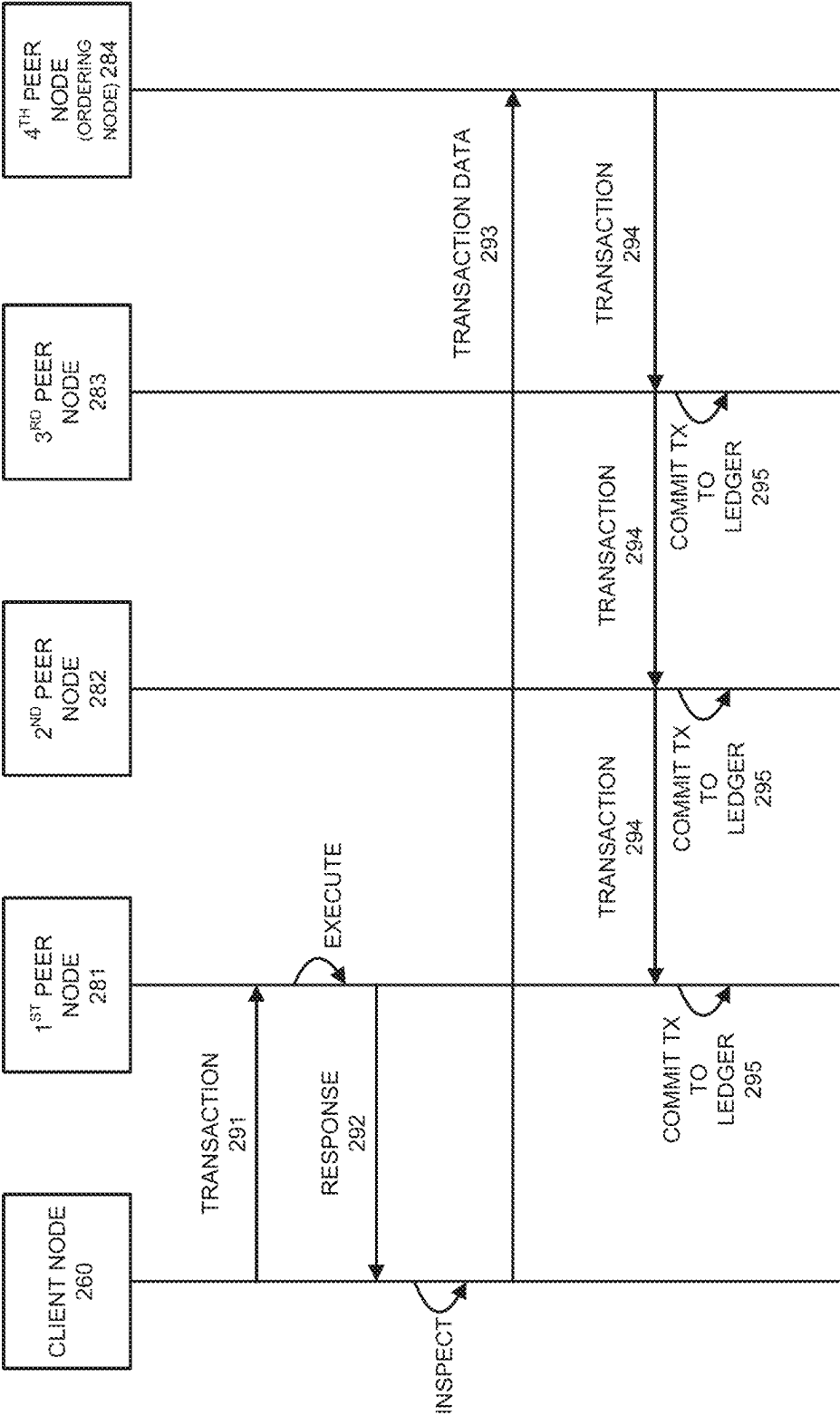


FIG. 2B

300

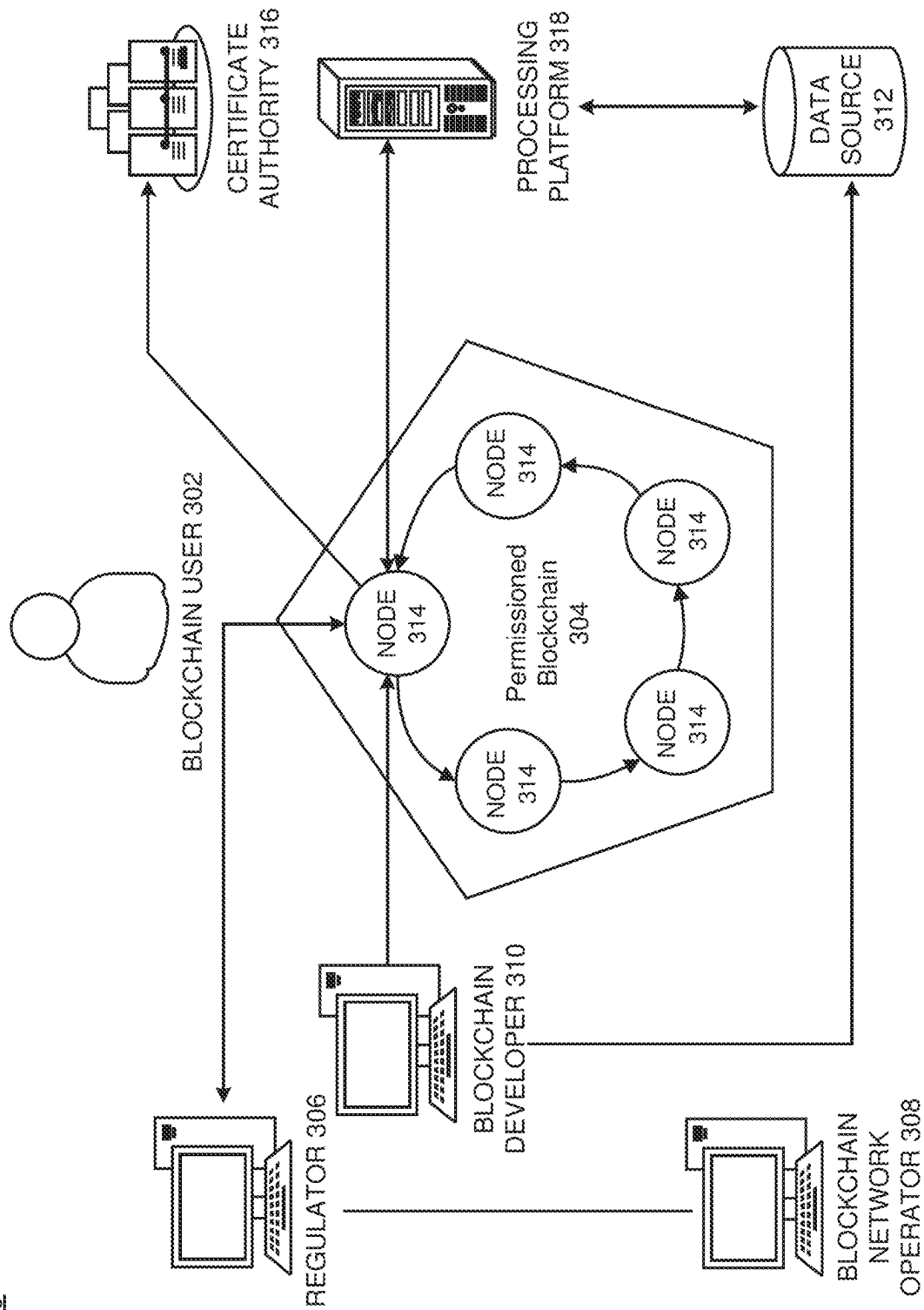
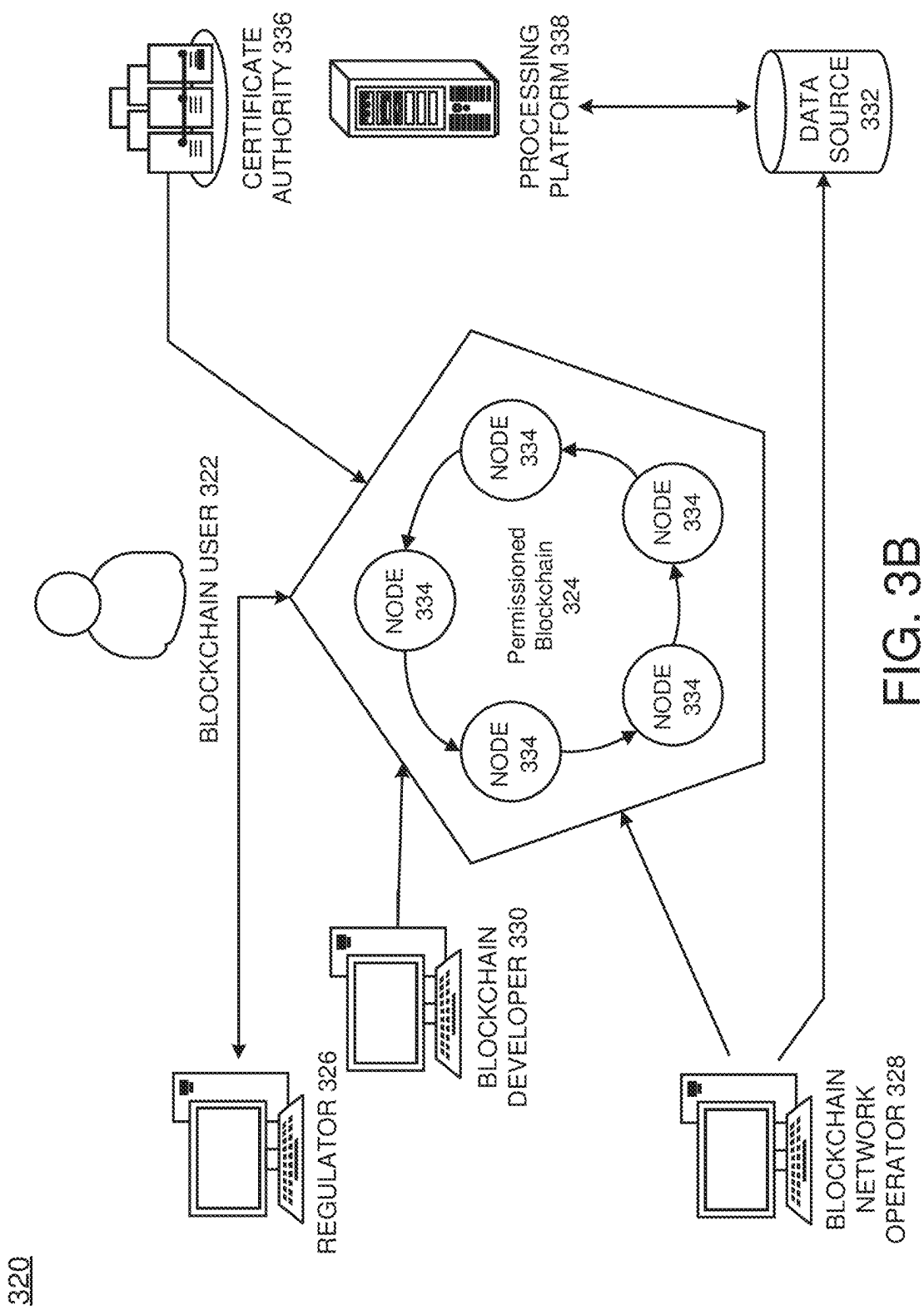


FIG. 3A



350

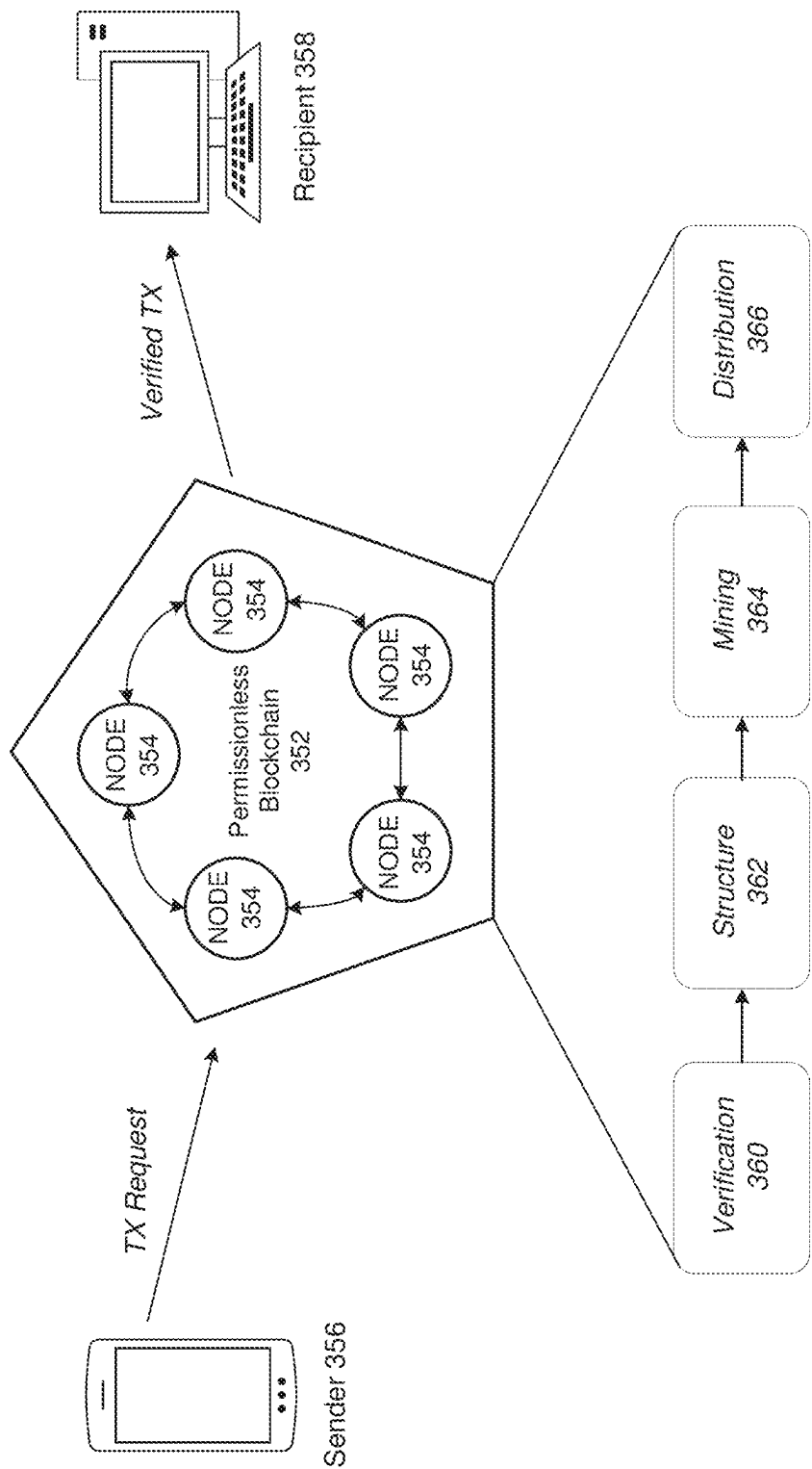
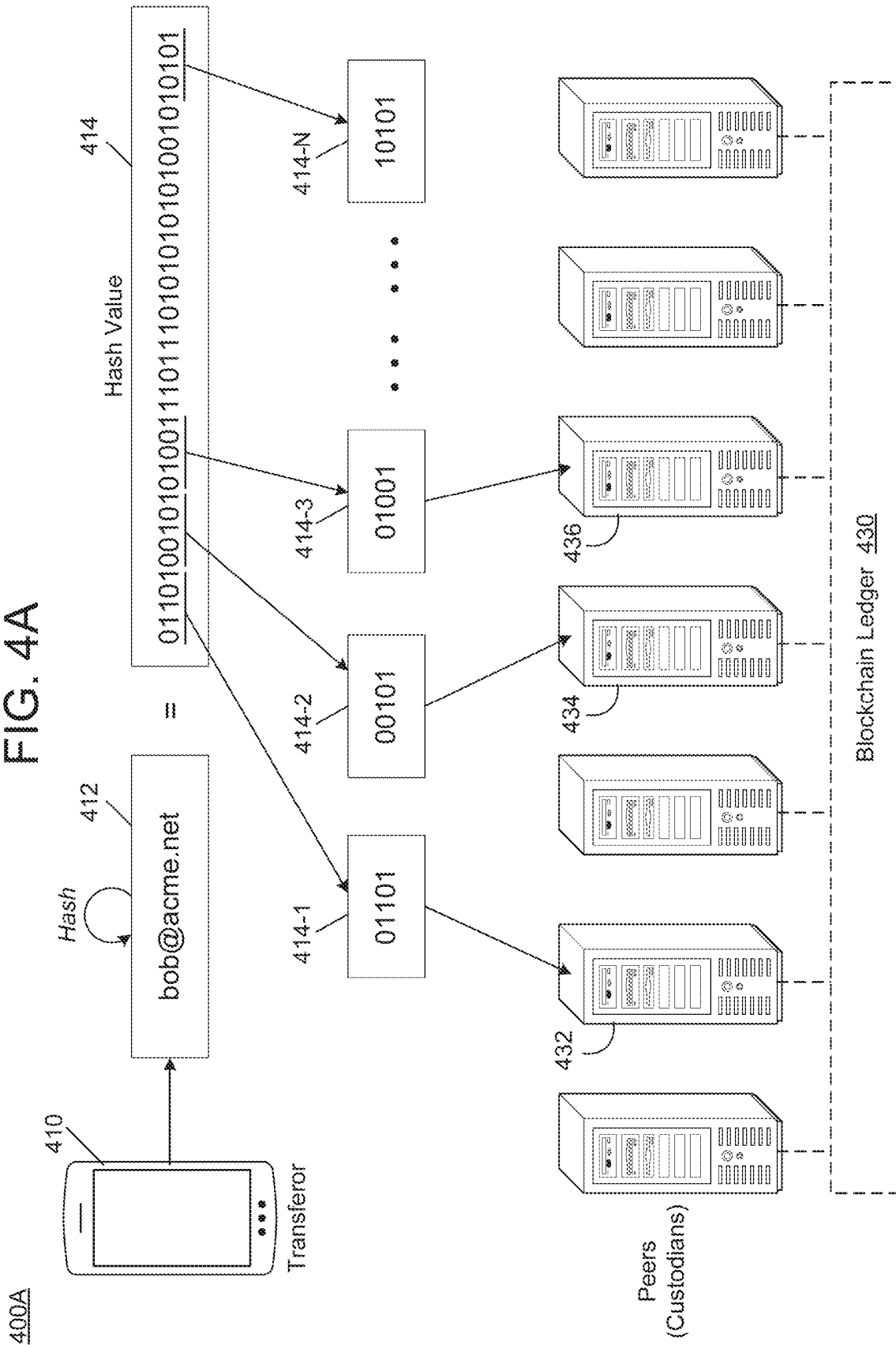


FIG. 3C

FIG. 4A



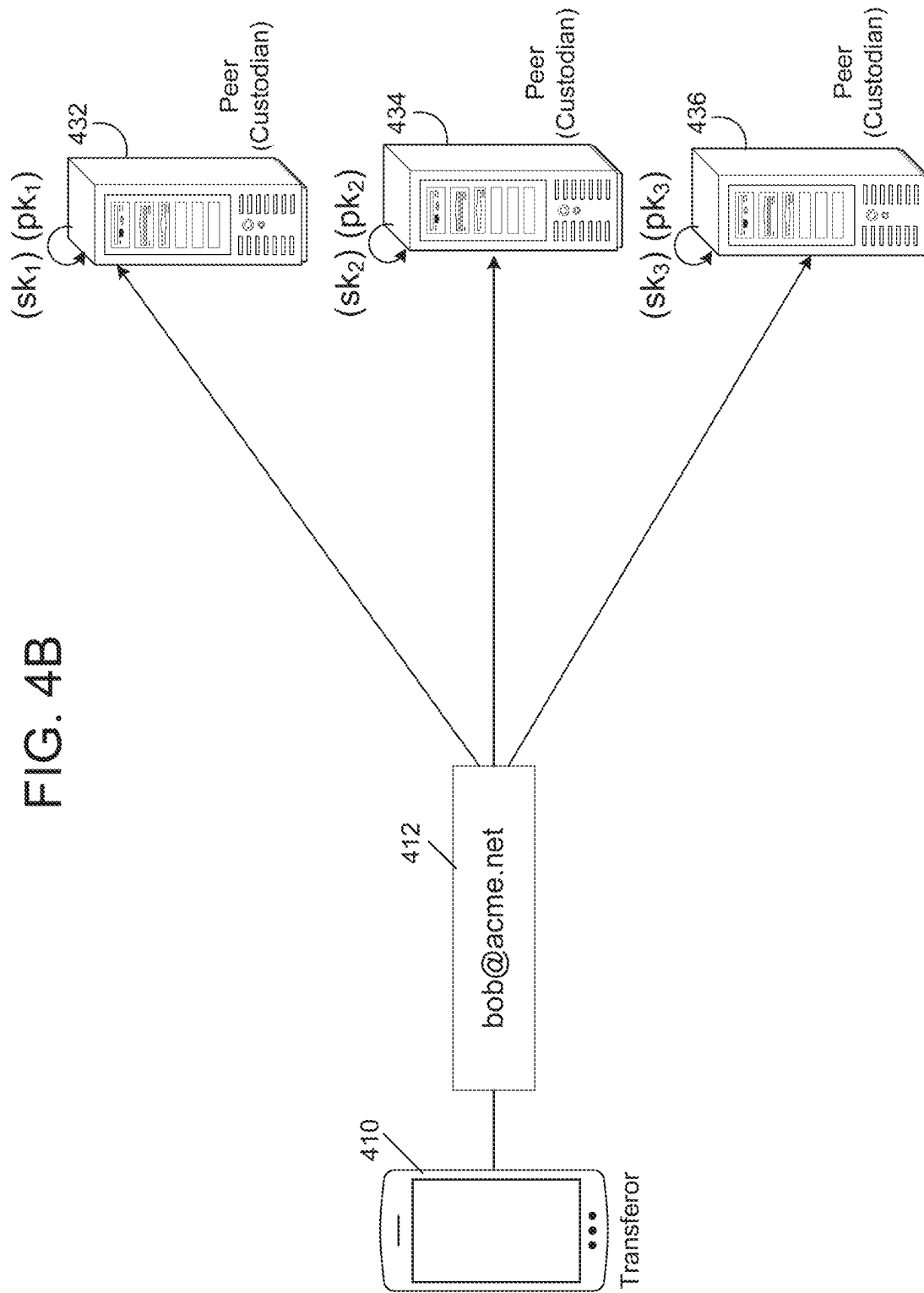
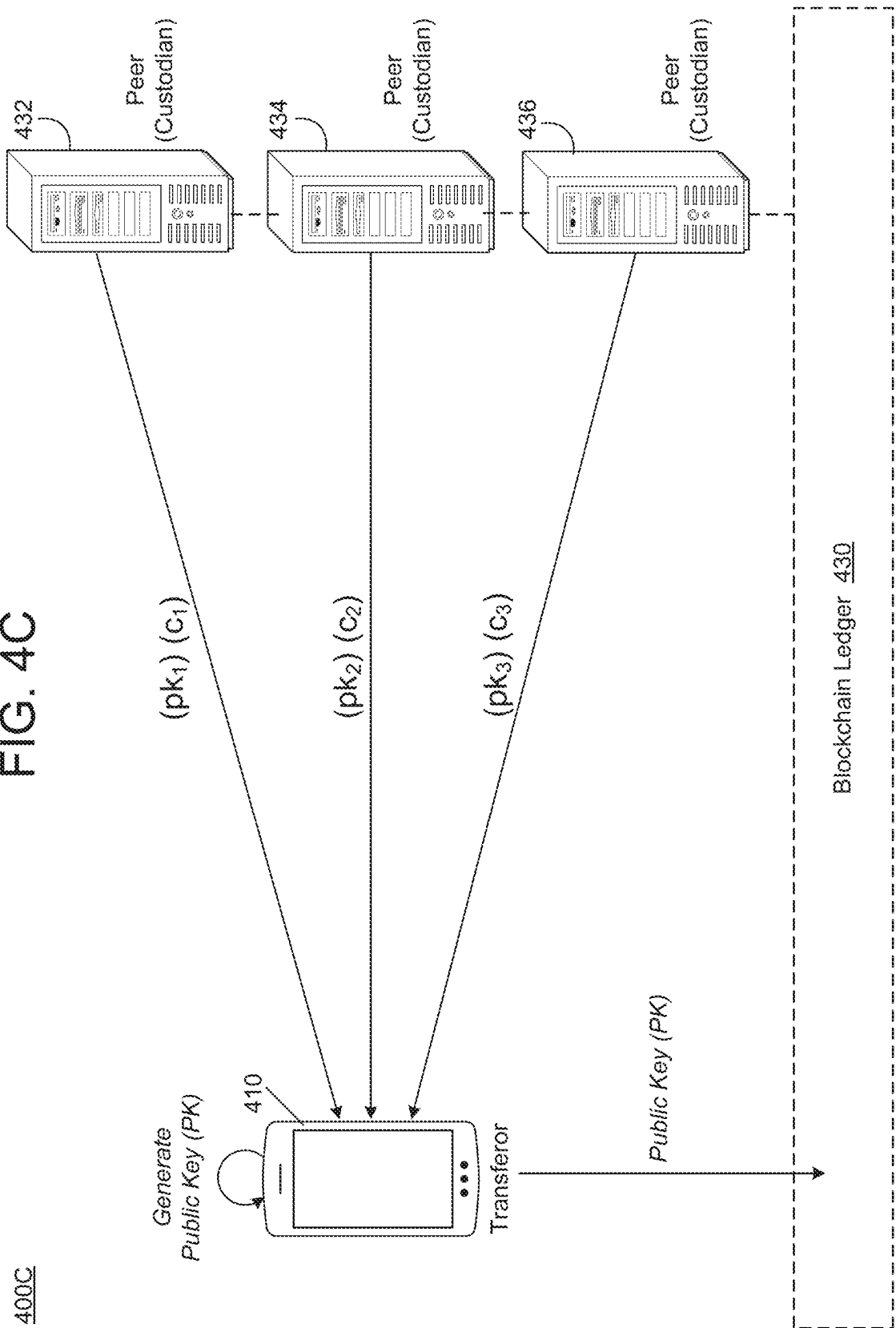


FIG. 4B

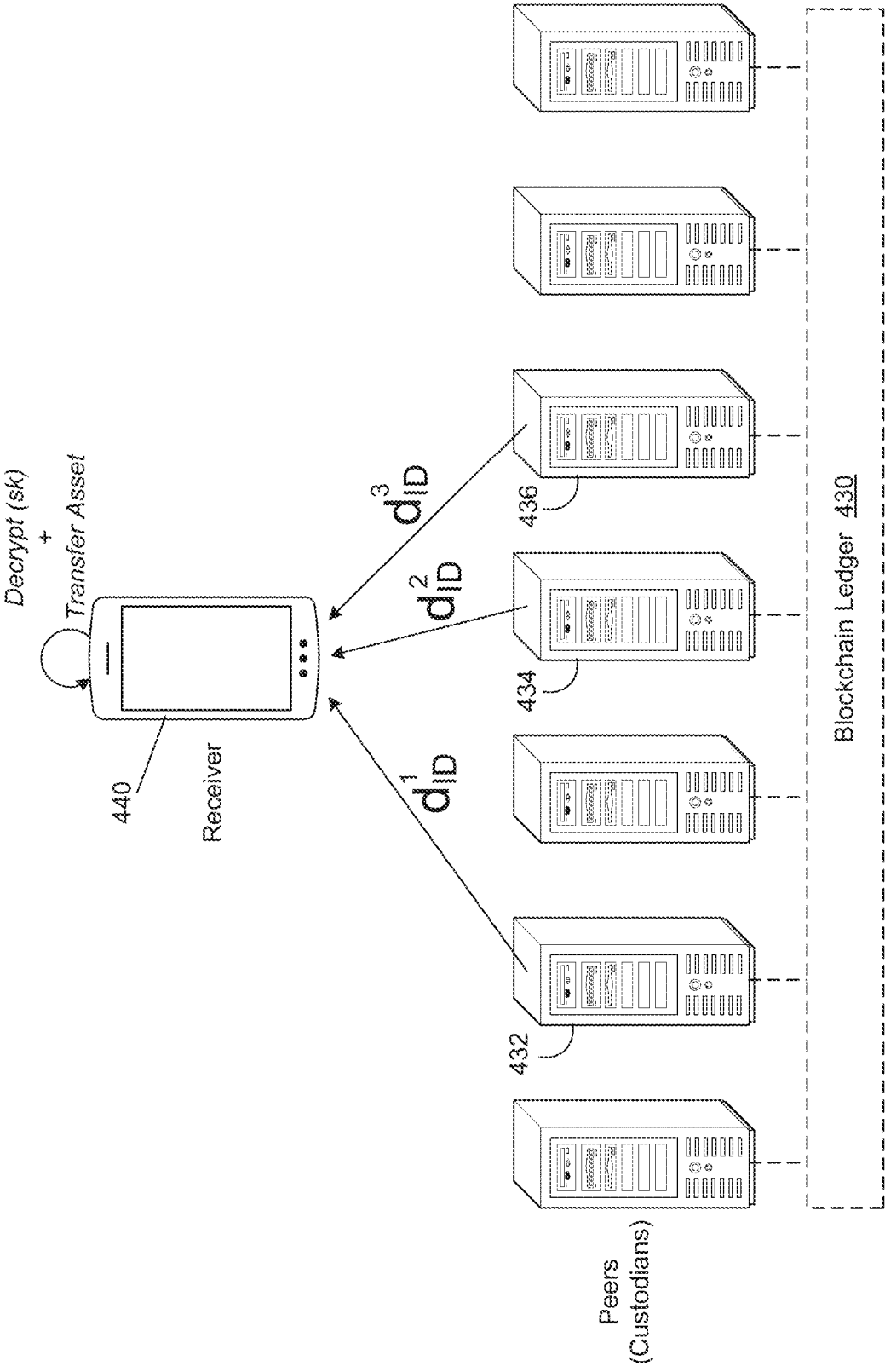
400B

FIG. 4C



400D

FIG. 4D



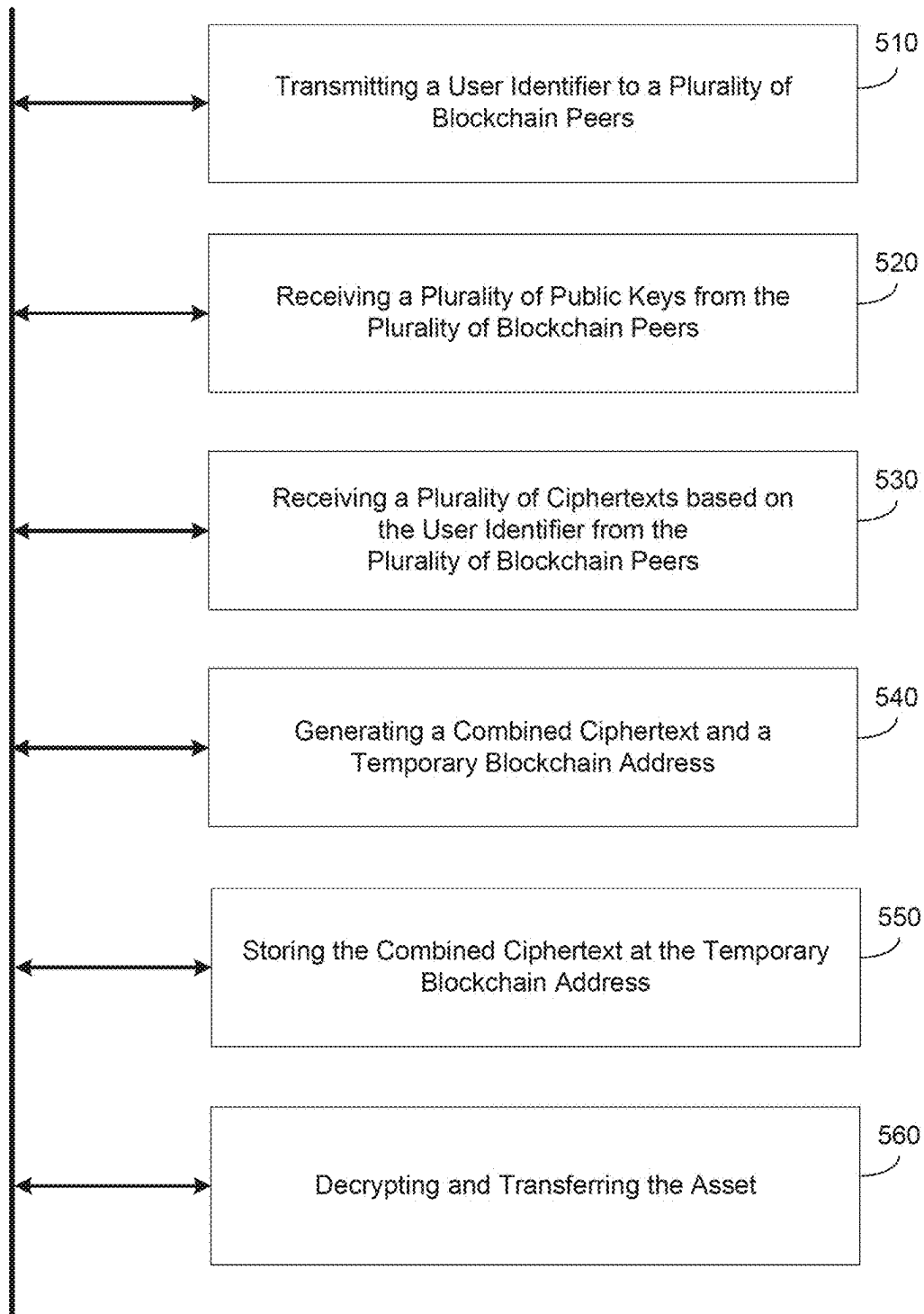
500

FIG. 5

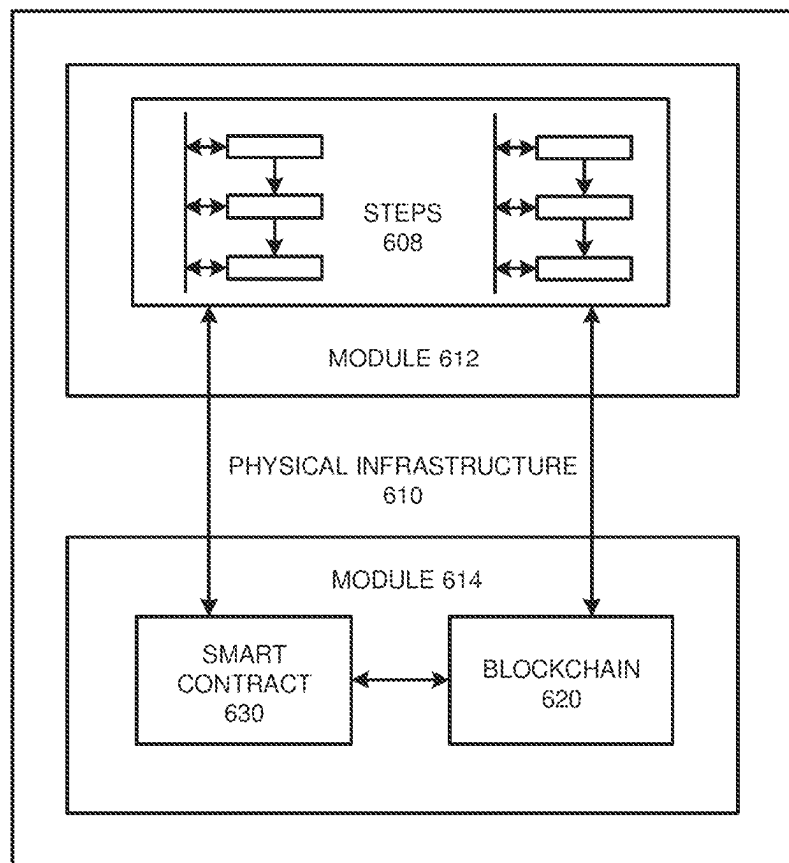
600

FIG. 6A

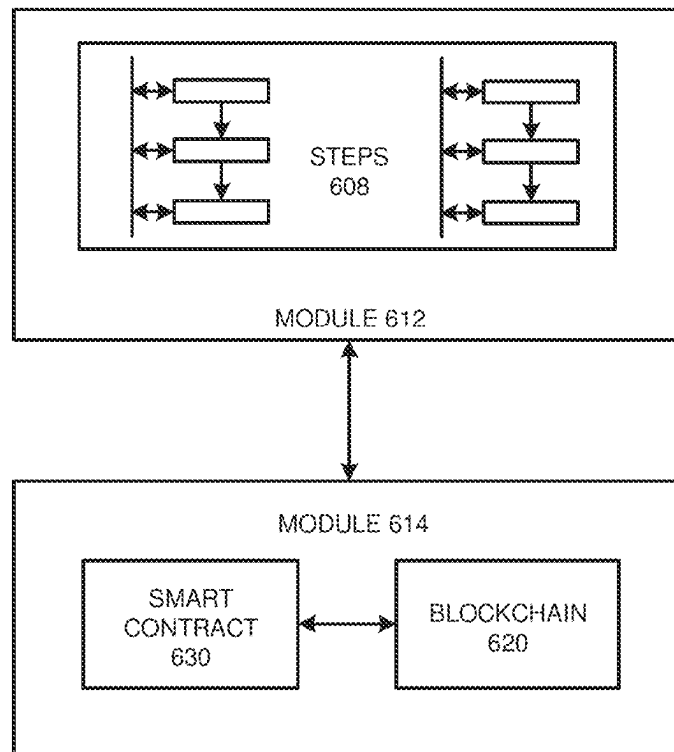
640

FIG. 6B

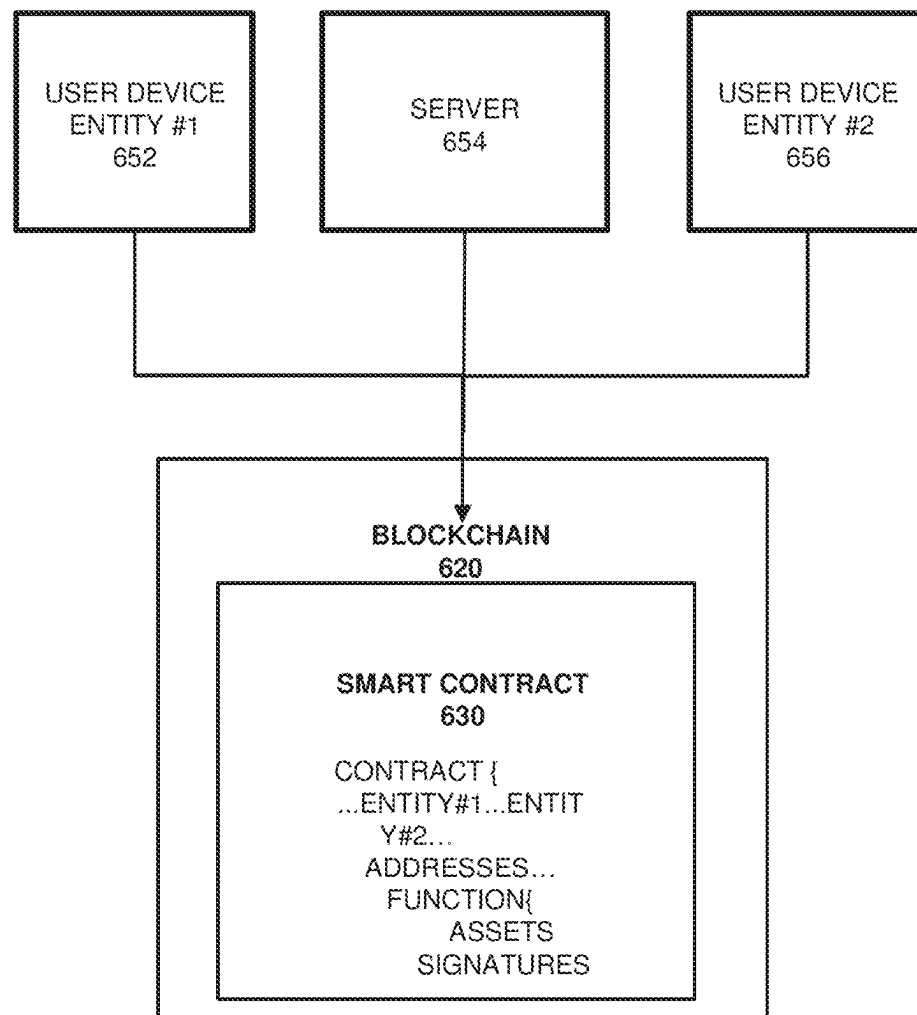
650

FIG. 6C

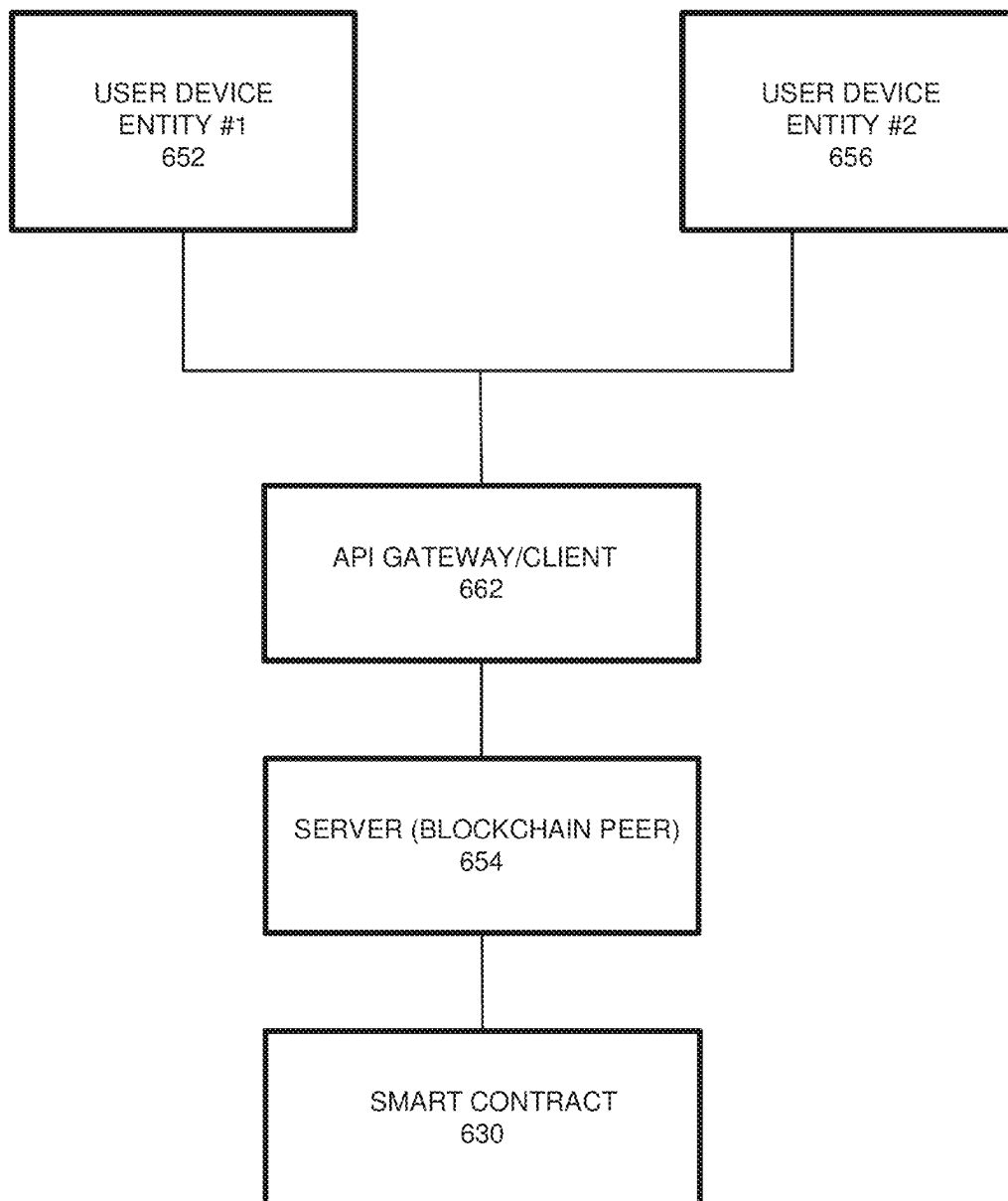
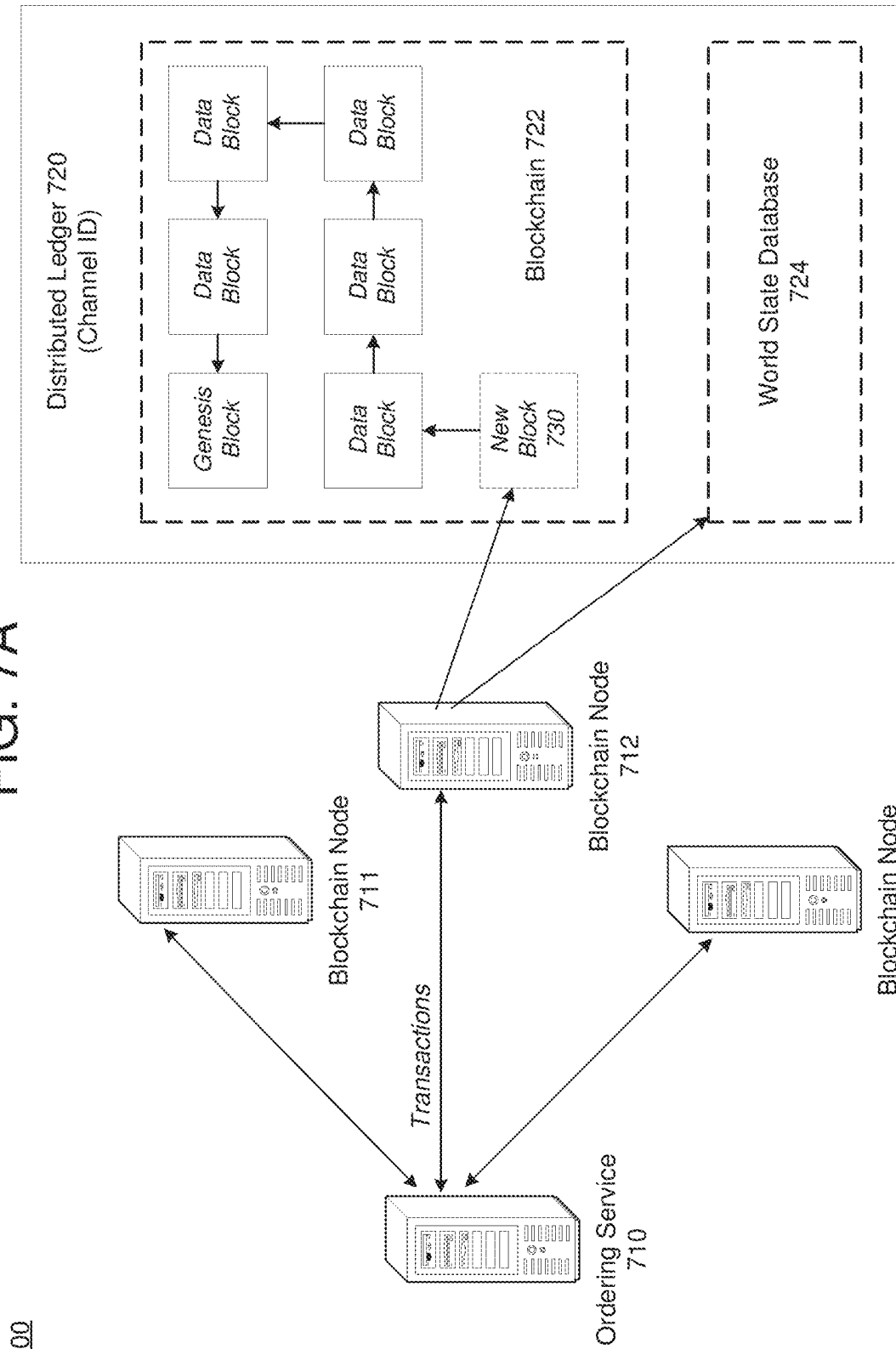
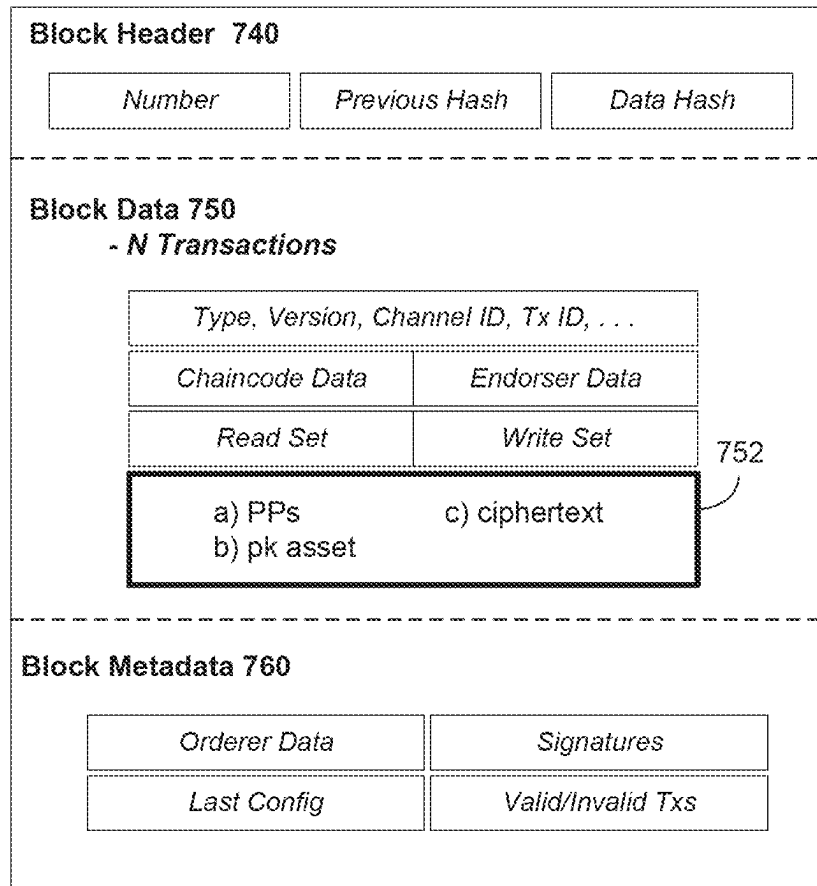
660

FIG. 6D

FIG. 7A



700

New Data Block 730**FIG. 7B**

770

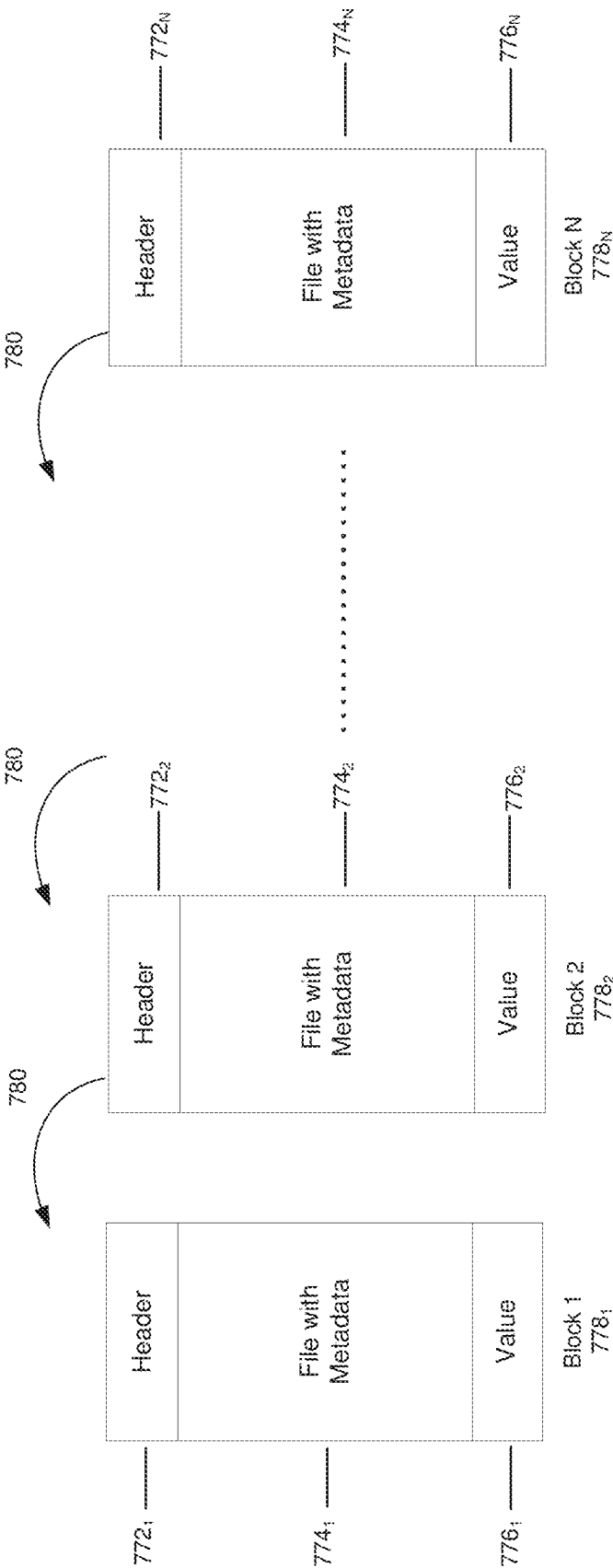


FIG. 7C

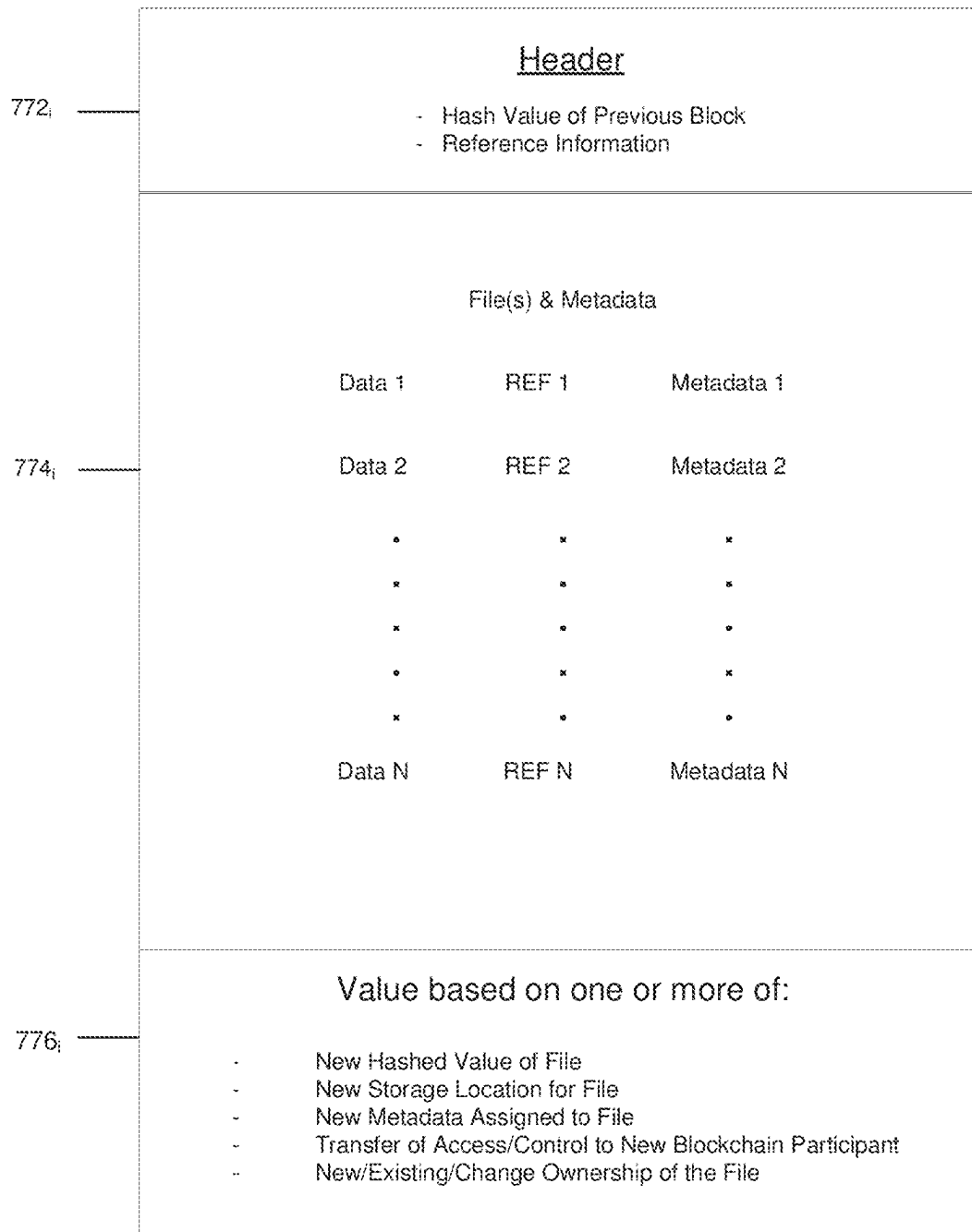
790Block_i

FIG. 7D

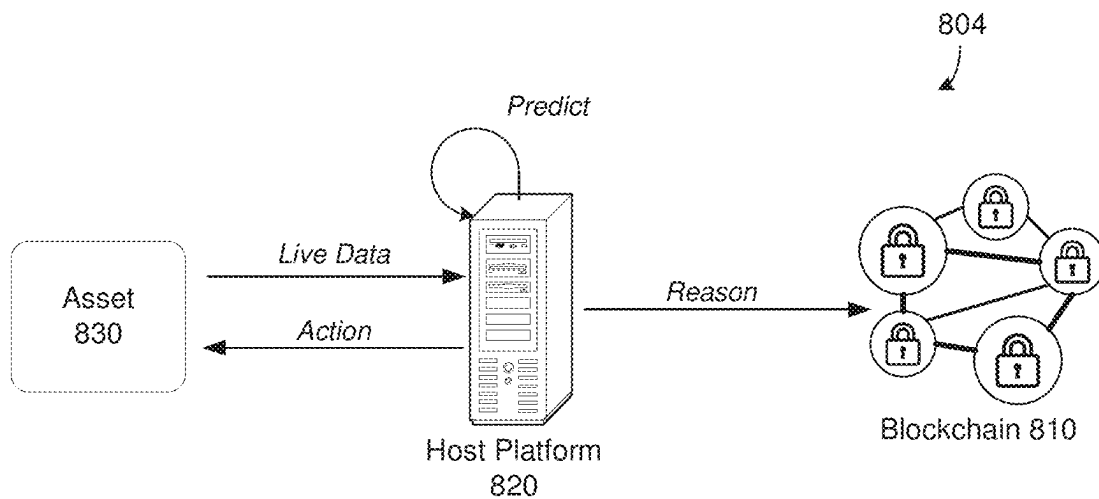
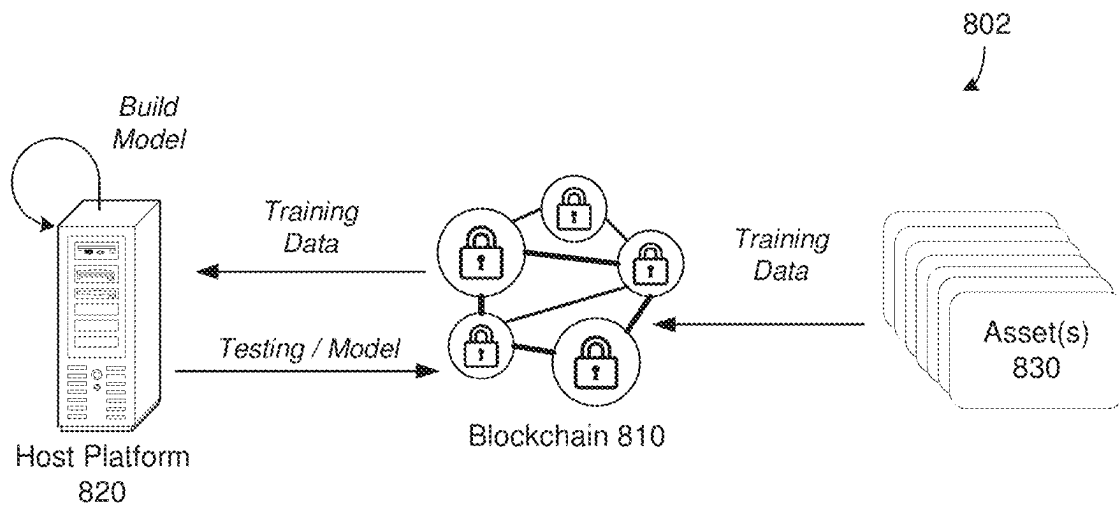
800

FIG. 8A

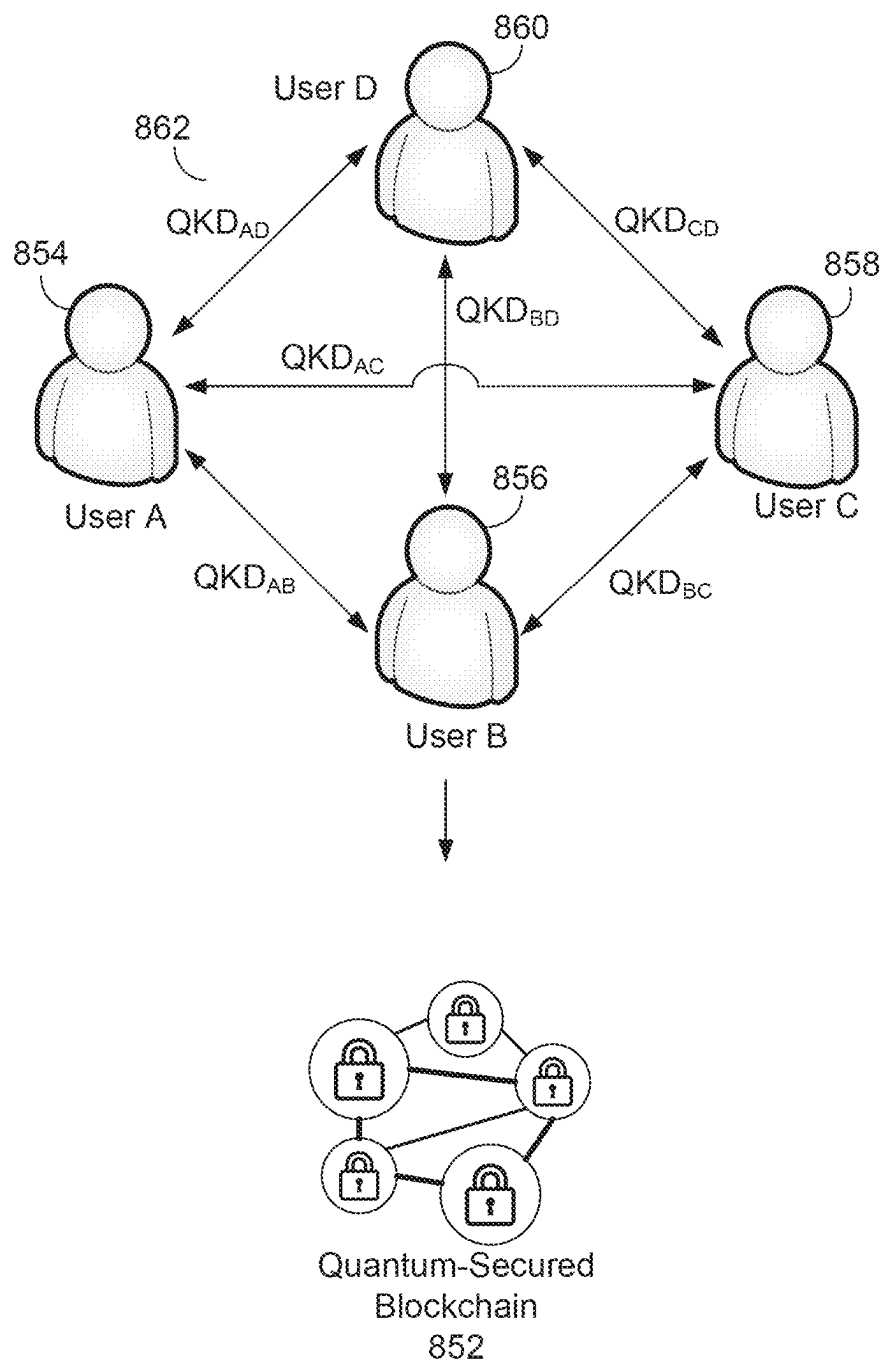
850

FIG. 8B

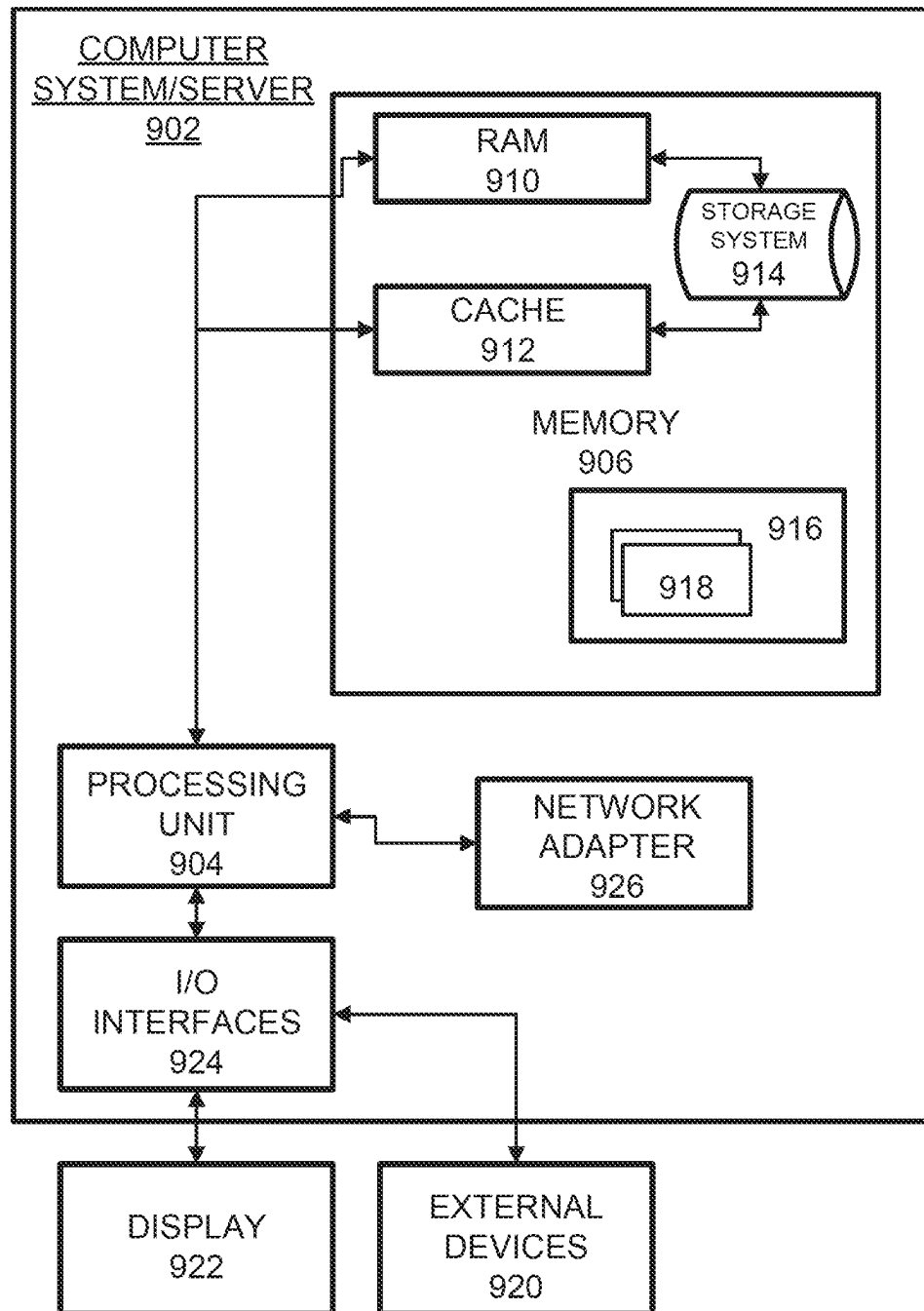
900

FIG. 9

1

FUTURE ASSET RECLAMATION VIA BLOCKCHAIN

BACKGROUND

A centralized platform stores and maintains data in a single location. This location is often a central computer, for example, a cloud computing environment, a web server, a mainframe computer, or the like. Information stored on a centralized platform is typically accessible from multiple different points. Multiple users or client workstations can work simultaneously on the centralized platform, for example, based on a client/server configuration. A centralized platform is easy to manage, maintain, and control, especially for purposes of security because of its single location. Within a centralized platform, data redundancy is minimized as a single storing place of all data also implies that a given set of data only has one primary record.

SUMMARY

One example embodiment provides an apparatus that includes a network interface configured to transmit a user identifier to a plurality of blockchain peers of a blockchain network, and a processor configured to one or more of receive a plurality of public keys from the plurality of blockchain peers, receive a plurality of ciphertexts from the plurality of blockchain peers generated based on the user identifier and a corresponding plurality of private keys of the plurality of blockchain peers, generate a combined ciphertext that includes the plurality of ciphertexts and generating a temporary blockchain address of the blockchain network based on the plurality of public keys, and store the combined ciphertext and a transfer of an asset via the temporary blockchain address of the blockchain network.

Another example embodiment provides a method that includes one or more of transmitting a user identifier to a plurality of blockchain peers of a blockchain network, receiving a plurality of public keys from the plurality of blockchain peers, receiving a plurality of ciphertexts from the plurality of blockchain peers generated based on the user identifier and a corresponding plurality of private keys of the plurality of blockchain peers, generating a combined ciphertext that includes the plurality of ciphertexts and generating a temporary blockchain address of the blockchain network based on the plurality of public keys, and storing the combined ciphertext and a transfer of an asset via the temporary blockchain address of the blockchain network.

A further example embodiment provides a computer-readable storage medium comprising instructions, that when read by a processor, cause the processor to perform one or more of transmitting a user identifier to a plurality of blockchain peers of a blockchain network, receiving a plurality of public keys from the plurality of blockchain peers, receiving a plurality of ciphertexts from the plurality of blockchain peers generated based on the user identifier and a corresponding plurality of private keys of the plurality of blockchain peers, generating a combined ciphertext that includes the plurality of ciphertexts and generating a temporary blockchain address of the blockchain network based on the plurality of public keys, and storing the combined ciphertext and a transfer of an asset via the temporary blockchain address of the blockchain network.

BRIEF DESCRIPTION OF THE DRAWINGS

FIG. 1A is a diagram illustrating a process of a blockchain transfer among registered blockchain clients according to example embodiments.

2

FIG. 1B is a diagram illustrating a process of a blockchain transfer when one of the clients is not yet a participant of the blockchain according to example embodiments.

FIG. 2A is a diagram illustrating an example blockchain architecture configuration, according to example embodiments.

FIG. 2B is a diagram illustrating a blockchain transactional flow among nodes, according to example embodiments.

FIG. 3A is a diagram illustrating a permissioned network, according to example embodiments.

FIG. 3B is a diagram illustrating another permissioned network, according to example embodiments.

FIG. 3C is a diagram illustrating a permissionless network, according to example embodiments.

FIG. 4A-4C are diagrams illustrating a process of an identity-based encryption (IBE) scheme according to example embodiments.

FIG. 4D is a diagram illustrating a process of claiming the asset according to example embodiments.

FIG. 5 is a diagram illustrating a method of an identity-based encryption for a future transfer of an asset according to example embodiments.

FIG. 6A is a diagram illustrating an example system configured to perform one or more operations described herein, according to example embodiments.

FIG. 6B is a diagram illustrating another example system configured to perform one or more operations described herein, according to example embodiments.

FIG. 6C is a diagram illustrating a further example system configured to utilize a smart contract, according to example embodiments.

FIG. 6D is a diagram illustrating yet another example system configured to utilize a blockchain, according to example embodiments.

FIG. 7A is a diagram illustrating a process of a new block being added to a distributed ledger, according to example embodiments.

FIG. 7B is a diagram illustrating data contents of a new data block, according to example embodiments.

FIG. 7C is a diagram illustrating a blockchain for digital content, according to example embodiments.

FIG. 7D is a diagram illustrating a block which may represent the structure of blocks in the blockchain, according to example embodiments.

FIG. 8A is a diagram illustrating an example blockchain which stores machine learning (artificial intelligence) data, according to example embodiments.

FIG. 8B is a diagram illustrating an example quantum-secure blockchain, according to example embodiments.

FIG. 9 is a diagram illustrating an example system that supports one or more of the example embodiments.

DETAILED DESCRIPTION

It will be readily understood that the instant components, as generally described and illustrated in the figures herein, may be arranged and designed in a wide variety of different configurations. Thus, the following detailed description of the embodiments of at least one of a method, apparatus, non-transitory computer readable medium and system, as represented in the attached figures, is not intended to limit the scope of the application as claimed but is merely representative of selected embodiments.

The instant features, structures, or characteristics as described throughout this specification may be combined or removed in any suitable manner in one or more embodi-

ments. For example, the usage of the phrases “example embodiments”, “some embodiments”, or other similar language, throughout this specification refers to the fact that a particular feature, structure, or characteristic described in connection with the embodiment may be included in at least one embodiment. Thus, appearances of the phrases “example embodiments”, “in some embodiments”, “in other embodiments”, or other similar language, throughout this specification do not necessarily all refer to the same group of embodiments, and the described features, structures, or characteristics may be combined or removed in any suitable manner in one or more embodiments. Further, in the diagrams, any connection between elements can permit one-way and/or two-way communication even if the depicted connection is a one-way or two-way arrow. Also, any device depicted in the drawings can be a different device. For example, if a mobile device is shown sending information, a wired device could also be used to send the information.

In addition, while the term “message” may have been used in the description of embodiments, the application may be applied to many types of networks and data. Furthermore, while certain types of connections, messages, and signaling may be depicted in exemplary embodiments, the application is not limited to a certain type of connection, message, and signaling.

Example embodiments provide methods, systems, components, non-transitory computer readable media, devices, and/or networks, directed to an identity-based encryption (IBE) scheme for transferring an asset managed by a blockchain to a non-registered/non-party of the blockchain. The example embodiments use the IBE to create a temporary blockchain address for holding the transfer transaction until the non-registered party can onboard/install the blockchain software and claim the asset.

In one embodiment the application utilizes a decentralized database (such as a blockchain) that is a distributed storage system, which includes multiple nodes that communicate with each other. The decentralized database includes an append-only immutable data structure resembling a distributed ledger capable of maintaining records between mutually untrusted parties. The untrusted parties are referred to herein as peers or peer nodes. Each peer maintains a copy of the database records and no single peer can modify the database records without a consensus being reached among the distributed peers. For example, the peers may execute a consensus protocol to validate blockchain storage transactions, group the storage transactions into blocks, and build a hash chain over the blocks. This process forms the ledger by ordering the storage transactions, as is necessary, for consistency. In various embodiments, a permissioned and/or a permissionless blockchain can be used. In a public or permission-less blockchain, anyone can participate without a specific identity. Public blockchains can involve native cryptocurrency and use consensus based on various protocols such as Proof of Work (PoW). On the other hand, a permissioned blockchain database provides secure interactions among a group of entities which share a common goal but which do not fully trust one another, such as businesses that exchange funds, goods, information, and the like.

This application can utilize a blockchain that operates arbitrary, programmable logic, tailored to a decentralized storage scheme and referred to as “smart contracts” or “chaincodes.” In some cases, specialized chaincodes may exist for management functions and parameters which are referred to as system chaincode. The application can further utilize smart contracts that are trusted distributed applications which leverage tamper-proof properties of the block-

chain database and an underlying agreement between nodes, which is referred to as an endorsement or endorsement policy. Blockchain transactions associated with this application can be “endorsed” before being committed to the blockchain while transactions, which are not endorsed, are disregarded. An endorsement policy allows chaincode to specify endorsers for a transaction in the form of a set of peer nodes that are necessary for endorsement. When a client sends the transaction to the peers specified in the endorsement policy, the transaction is executed to validate the transaction. After validation, the transactions enter an ordering phase in which a consensus protocol is used to produce an ordered sequence of endorsed transactions grouped into blocks.

This application can utilize nodes that are the communication entities of the blockchain system. A “node” may perform a logical function in the sense that multiple nodes of different types can run on the same physical server. Nodes are grouped in trust domains and are associated with logical entities that control them in various ways. Nodes may include different types, such as a client or submitting-client node which submits a transaction-invocation to an endorser (e.g., peer), and broadcasts transaction-proposals to an ordering service (e.g., ordering node). Another type of node is a peer node which can receive client submitted transactions, commit the transactions and maintain a state and a copy of the ledger of blockchain transactions. Peers can also have the role of an endorser, although it is not a requirement. An ordering-service-node or orderer is a node running the communication service for all nodes, and which implements a delivery guarantee, such as a broadcast to each of the peer nodes in the system when committing transactions and modifying a world state of the blockchain, which is another name for the initial blockchain transaction which normally includes control and setup information.

This application can utilize a ledger that is a sequenced, tamper-resistant record of all state transitions of a blockchain. State transitions may result from chaincode invocations (i.e., transactions) submitted by participating parties (e.g., client nodes, ordering nodes, endorser nodes, peer nodes, etc.). Each participating party (such as a peer node) can maintain a copy of the ledger. A transaction may result in a set of asset key-value pairs being committed to the ledger as one or more operands, such as creates, updates, deletes, and the like. The ledger includes a blockchain (also referred to as a chain) which is used to store an immutable, sequenced record in blocks. The ledger also includes a state database which maintains a current state of the blockchain.

This application can utilize a chain that is a transaction log which is structured as hash-linked blocks, and each block contains a sequence of N transactions where N is equal to or greater than one. The block header includes a hash of the block’s transactions, as well as a hash of the prior block’s header. In this way, all transactions on the ledger may be sequenced and cryptographically linked together. Accordingly, it is not possible to tamper with the ledger data without breaking the hash links. A hash of a most recently added blockchain block represents every transaction on the chain that has come before it, making it possible to ensure that all peer nodes are in a consistent and trusted state. The chain may be stored on a peer node file system (i.e., local, attached storage, cloud, etc.), efficiently supporting the append-only nature of the blockchain workload.

The current state of the immutable ledger represents the latest values for all keys that are included in the chain transaction log. Since the current state represents the latest key values known to a channel, it is sometimes referred to

as a world state. Chaincode invocations execute transactions against the current state data of the ledger. To make these chaincode interactions efficient, the latest values of the keys may be stored in a state database. The state database may be simply an indexed view into the chain's transaction log, it can therefore be regenerated from the chain at any time. The state database may automatically be recovered (or generated if needed) upon peer node startup, and before transactions are accepted.

In the example embodiments, a blockchain client can transfer an asset (managed by the blockchain) to a party (e.g., a future client of the blockchain) that is not yet a participant of the blockchain. For example, the transferor client can transfer the asset to a temporary holding area/network location generated from an identity-based encryption scheme from the identity of the party. Furthermore, the party can subsequently claim the asset once it onboarded itself to the blockchain network based on the identity of the party. The example embodiments speed-up the process of a blockchain transfer of assets when a party to the transaction is not yet registered with or able to participate in the blockchain network.

FIG. 1A illustrates a process 100A of a blockchain transfer among registered blockchain clients 110 and 130 according to example embodiments. Referring to FIG. 1A, the client 110 (transferor) submits a transaction to a blockchain 120 with instructions for transferring an asset (managed by the blockchain 120) to the client 130 (receiver). Here, both the client 110 and the client 130 are already registered members of the blockchain 120. That is, both the transferor and the receiver have installed blockchain client software for interacting with blockchain peers that manage the blockchain 120. Thus, the blockchain transaction with the transfer of the asset from the client 110 to the client 130 can be performed immediately/real-time because both parties are already members of the blockchain and can be identified based on keys that are uploaded to the blockchain 120 during a registration process.

For example, during a registration process, the client 130 may generate a private/public key pair and submit the public key to the blockchain 120. In response, the client 130 may receive a credential identified by the public key. This information is then made available to other participants of the blockchain 120. Accordingly, when the client 110 wants to transfer the asset to the client 130, the client 110 changes the ownership of the asset on the blockchain 120 to the identifier of the credential of the client 130.

However, this process requires the client 130 (i.e., the receiver of the transfer of the asset managed by the blockchain 120) to be a participating member of the blockchain 120. This usually entails a registration process that requires the participating member to divulge a public key to the blockchain network. The problem occurs when a receiver (such as the purchaser of a house, car, digital asset, etc.) is not an onboarded party to the blockchain 120. Traditionally, such a receiver cannot receive a transaction via the blockchain because there is no mechanism for transferring an asset via the blockchain to a client that is not a participant of the blockchain. The example embodiments overcome this drawback in the art.

FIG. 1B illustrates a process 100B of a blockchain transfer when one of the clients (i.e., client 140) is not yet a participant of the blockchain according to example embodiments. According to various embodiments, a temporary blockchain address 122 (e.g., network address, public key, etc.) can be generated via an identity-based encryption scheme that is homomorphic (there exists an operator (either

addition or multiplication) such that when operands of the scheme are operated on with the operator, their structure is preserved). The asset transferred from the client 110 to the client 140 can temporarily be held/stored at the temporary blockchain address 122. That is, a first transaction can be used to transfer the asset from an address of the client 110 to the temporary blockchain address 122. In this way, the temporary blockchain address 122 becomes the temporary owner of the transferred asset until the client 140 has onboarded to the blockchain 120.

When the client 140 subsequently onboarded to the blockchain 120, the client 140 can obtain the temporary address 122 from the custodians based on the user identifier of the client 140 used to select the custodians. Furthermore, the client 140 can transfer the asset from the temporary blockchain address 122 to a blockchain address of the onboarded client 140 via a second blockchain transaction.

The process of creating the public key used to form the temporary blockchain address 122 is further described in the examples of FIGS. 4A-4D. Here, an identifier (user identifier) of the client 140 can be used by the client 110 (transferor) to randomly select a subset of "custodians" of the temporary asset transfer until the client 140 is onboarded. The randomly selected "custodians" may be blockchain peers that co-manage the blockchain 120 and which are selected using a pseudo-random algorithm based on a value that is publicly available to the other participants of the blockchain 120. In particular, the client 110 may hash the user identifier of the client 140 (e.g., email address, phone number, license number, passport ID, etc.) to create a hashed string/binary string of a fixed length.

Next, the client 110 may iteratively slice off chunks/subsets of characters from the string starting with one end of the string and working towards the other end. Each sliced subset can be used to map to a peer ID in a mapping table or something similar. For example, Peer 1 may be mapped to a range of binary values from (0000-0100). As another example, Peer 2 may be mapped to a range of binary values such as (0101-0111), etc. There are many ways that this mapping process may be performed and the embodiments herein should not be limited to any particular mappings scheme. The idea is to use the user identity of client 140 (which both the client 110 and the client 140 know) to enable the client 140 to claim the asset transfer at a subsequent time.

FIG. 2A illustrates a blockchain architecture configuration 200, according to example embodiments. Referring to FIG. 2A, the blockchain architecture 200 may include certain blockchain elements, for example, a group of blockchain nodes 202. The blockchain nodes 202 may include one or more nodes 204-210 (these four nodes are depicted by example only). These nodes participate in a number of activities, such as blockchain transaction addition and validation process (consensus). One or more of the blockchain nodes 204-210 may endorse transactions based on endorsement policy and may provide an ordering service for all blockchain nodes in the architecture 200. A blockchain node may initiate a blockchain authentication and seek to write to a blockchain immutable ledger stored in blockchain layer 216, a copy of which may also be stored on the underpinning physical infrastructure 214. The blockchain configuration may include one or more applications 224 which are linked to application programming interfaces (APIs) 222 to access and execute stored program/application code 220 (e.g., chaincode, smart contracts, etc.) which can be created according to a customized configuration sought by participants and can maintain their own state, control their own assets, and receive external information. This can be

deployed as a transaction and installed, via appending to the distributed ledger, on all blockchain nodes **204-210**.

The blockchain base or platform **212** may include various layers of blockchain data, services (e.g., cryptographic trust services, virtual execution environment, etc.), and underpinning physical computer infrastructure that may be used to receive and store new transactions and provide access to auditors which are seeking to access data entries. The blockchain layer **216** may expose an interface that provides access to the virtual execution environment necessary to process the program code and engage the physical infrastructure **214**. Cryptographic trust services **218** may be used to verify transactions such as asset exchange transactions and keep information private.

The blockchain architecture configuration of FIG. 2A may process and execute program/application code **220** via one or more interfaces exposed, and services provided, by blockchain platform **212**. The code **220** may control blockchain assets. For example, the code **220** can store and transfer data, and may be executed by nodes **204-210** in the form of a smart contract and associated chaincode with conditions or other code elements subject to its execution. As a non-limiting example, smart contracts may be created to execute reminders, updates, and/or other notifications subject to the changes, updates, etc. The smart contracts can themselves be used to identify rules associated with authorization and access requirements and usage of the ledger. For example, the smart contract (or chaincode executing the logic of the smart contract) may read blockchain data **226** which may be processed by one or more processing entities (e.g., virtual machines) included in the blockchain layer **216** to generate results **228** including alerts, determining liability, and the like, within a complex service scenario. The physical infrastructure **214** may be utilized to retrieve any of the data or information described herein.

A smart contract may be created via a high-level application and programming language, and then written to a block in the blockchain. The smart contract may include executable code which is registered, stored, and/or replicated with a blockchain (e.g., distributed network of blockchain peers). A transaction is an execution of the smart contract code which can be performed in response to conditions associated with the smart contract being satisfied. The executing of the smart contract may trigger a trusted modification(s) to a state of a digital blockchain ledger. The modification(s) to the blockchain ledger caused by the smart contract execution may be automatically replicated throughout the distributed network of blockchain peers through one or more consensus protocols.

The smart contract may write data to the blockchain in the format of key-value pairs. Furthermore, the smart contract code can read the values stored in a blockchain and use them in application operations. The smart contract code can write the output of various logic operations into one or more blocks within the blockchain. The code may be used to create a temporary data structure in a virtual machine or other computing platform. Data written to the blockchain can be public and/or can be encrypted and maintained as private. The temporary data that is used/generated by the smart contract is held in memory by the supplied execution environment, then deleted once the data needed for the blockchain is identified.

A chaincode may include the code interpretation of a smart contract. For example, the chaincode may include a packaged and deployable version of the logic within the smart contract. As described herein, the chaincode may be program code deployed on a computing network, where it is

executed and validated by chain validators together during a consensus process. The chaincode may receive a hash and retrieve from the blockchain a hash associated with the data template created by use of a previously stored feature extractor. If the hashes of the hash identifier and the hash created from the stored identifier template data match, then the chaincode sends an authorization key to the requested service. The chaincode may write to the blockchain data associated with the cryptographic details.

FIG. 2B illustrates an example of a blockchain transactional flow **250** between nodes of the blockchain in accordance with an example embodiment. Referring to FIG. 2B, the transaction flow may include a client node **260** transmitting a transaction proposal **291** to an endorsing peer node **281**. The endorsing peer **281** may verify the client signature and execute a chaincode function to initiate the transaction. The output may include the chaincode results, a set of key/value versions that were read in the chaincode (read set), and the set of keys/values that were written in chaincode (write set). Here, the endorsing peer **281** may determine whether or not to endorse the transaction proposal. The proposal response **292** is sent back to the client **260** along with an endorsement signature, if approved. The client **260** assembles the endorsements into a transaction payload **293** and broadcasts it to an ordering service node **284**. The ordering service node **284** then delivers ordered transactions as blocks to all peers **281-283** on a channel. Before commitment to the blockchain, each peer **281-283** may validate the transaction. For example, the peers may check the endorsement policy to ensure that the correct allotment of the specified peers have signed the results and authenticated the signatures against the transaction payload **293**.

Referring again to FIG. 2B, the client node initiates the transaction **291** by constructing and sending a request to the peer node **281**, which is an endorser. The client **260** may include an application leveraging a supported software development kit (SDK), which utilizes an available API to generate a transaction proposal. The proposal is a request to invoke a chaincode function so that data can be read and/or written to the ledger (i.e., write new key value pairs for the assets). The SDK may serve as a shim to package the transaction proposal into a properly architected format (e.g., protocol buffer over a remote procedure call (RPC)) and take the client's cryptographic credentials to produce a unique signature for the transaction proposal.

In response, the endorsing peer node **281** may verify (a) that the transaction proposal is well formed, (b) the transaction has not been submitted already in the past (replay-attack protection), (c) the signature is valid, and (d) that the submitter (client **260**, in the example) is properly authorized to perform the proposed operation on that channel. The endorsing peer node **281** may take the transaction proposal inputs as arguments to the invoked chaincode function. The chaincode is then executed against a current state database to produce transaction results including a response value, read set, and write set. However, no updates are made to the ledger at this point. In **292**, the set of values, along with the endorsing peer node's **281** signature is passed back as a proposal response **292** to the SDK of the client **260** which parses the payload for the application to consume.

In response, the application of the client **260** inspects/verifies the endorsing peers signatures and compares the proposal responses to determine if the proposal response is the same. If the chaincode only queried the ledger, the application would inspect the query response and would typically not submit the transaction to the ordering node service **284**. If the client application intends to submit the

transaction to the ordering node service **284** to update the ledger, the application determines if the specified endorsement policy has been fulfilled before submitting (i.e., did all peer nodes necessary for the transaction endorse the transaction). Here, the client may include only one of multiple parties to the transaction. In this case, each client may have their own endorsing node, and each endorsing node will need to endorse the transaction. The architecture is such that even if an application selects not to inspect responses or otherwise forwards an unendorsed transaction, the endorsement policy will still be enforced by peers and upheld at the commit validation phase.

After successful inspection, in step **293** the client **260** assembles endorsements into a transaction proposal and broadcasts the transaction proposal and response within a transaction message to the ordering node **284**. The transaction may contain the read/write sets, the endorsing peers signatures and a channel ID. The ordering node **284** does not need to inspect the entire content of a transaction in order to perform its operation, instead the ordering node **284** may simply receive transactions from all channels in the network, order them chronologically by channel, and create blocks of transactions per channel.

The blocks are delivered from the ordering node **284** to all peer nodes **281-283** on the channel. The data section within the block may be validated to ensure an endorsement policy is fulfilled and to ensure that there have been no changes to ledger state for read set variables since the read set was generated by the transaction execution. Furthermore, in step **295** each peer node **281-283** appends the block to the channel's chain, and for each valid transaction the write sets are committed to current state database. An event may be emitted, to notify the client application that the transaction (invocation) has been immutably appended to the chain, as well as to notify whether the transaction was validated or invalidated.

FIG. 3A illustrates an example of a permissioned blockchain network **300**, which features a distributed, decentralized peer-to-peer architecture. In this example, a blockchain user **302** may initiate a transaction to the permissioned blockchain **304**. In this example, the transaction can be a deploy, invoke, or query, and may be issued through a client-side application leveraging an SDK, directly through an API, etc. Networks may provide access to a regulator **306**, such as an auditor. A blockchain network operator **308** manages member permissions, such as enrolling the regulator **306** as an "auditor" and the blockchain user **302** as a "client". An auditor could be restricted only to querying the ledger whereas a client could be authorized to deploy, invoke, and query certain types of chaincode.

A blockchain developer **310** can write chaincode and client-side applications. The blockchain developer **310** can deploy chaincode directly to the network through an interface. To include credentials from a traditional data source **312** in chaincode, the developer **310** could use an out-of-band connection to access the data. In this example, the blockchain user **302** connects to the permissioned blockchain **304** through a peer node **314**. Before proceeding with any transactions, the peer node **314** retrieves the user's enrollment and transaction certificates from a certificate authority **316**, which manages user roles and permissions. In some cases, blockchain users must possess these digital certificates in order to transact on the permissioned blockchain **304**. Meanwhile, a user attempting to utilize chaincode may be required to verify their credentials on the traditional data source **312**. To confirm the user's authorization, chain-

code can use an out-of-band connection to this data through a traditional processing platform **318**.

FIG. 3B illustrates another example of a permissioned blockchain network **320**, which features a distributed, decentralized peer-to-peer architecture. In this example, a blockchain user **322** may submit a transaction to the permissioned blockchain **324**. In this example, the transaction can be a deploy, invoke, or query, and may be issued through a client-side application leveraging an SDK, directly through an API, etc. Networks may provide access to a regulator **326**, such as an auditor. A blockchain network operator **328** manages member permissions, such as enrolling the regulator **326** as an "auditor" and the blockchain user **322** as a "client". An auditor could be restricted only to querying the ledger whereas a client could be authorized to deploy, invoke, and query certain types of chaincode.

A blockchain developer **330** writes chaincode and client-side applications. The blockchain developer **330** can deploy chaincode directly to the network through an interface. To include credentials from a traditional data source **332** in chaincode, the developer **330** could use an out-of-band connection to access the data. In this example, the blockchain user **322** connects to the network through a peer node **334**. Before proceeding with any transactions, the peer node **334** retrieves the user's enrollment and transaction certificates from the certificate authority **336**. In some cases, blockchain users must possess these digital certificates in order to transact on the permissioned blockchain **324**. Meanwhile, a user attempting to utilize chaincode may be required to verify their credentials on the traditional data source **332**. To confirm the user's authorization, chaincode can use an out-of-band connection to this data through a traditional processing platform **338**.

In some embodiments, the blockchain herein may be a permissionless blockchain. In contrast with permissioned blockchains which require permission to join, anyone can join a permissionless blockchain. For example, to join a permissionless blockchain a user may create a personal address and begin interacting with the network, by submitting transactions, and hence adding entries to the ledger. Additionally, all parties have the choice of running a node on the system and employing the mining protocols to help verify transactions.

FIG. 3C illustrates a process **350** of a transaction being processed by a permissionless blockchain **352** including a plurality of nodes **354**. A sender **356** desires to send payment or some other form of value (e.g., a deed, medical records, a contract, a good, a service, or any other asset that can be encapsulated in a digital record) to a recipient **358** via the permissionless blockchain **352**. In one embodiment, each of the sender device **356** and the recipient device **358** may have digital wallets (associated with the blockchain **352**) that provide user interface controls and a display of transaction parameters. In response, the transaction is broadcast throughout the blockchain **352** to the nodes **354**. Depending on the blockchain's **352** network parameters the nodes verify **360** the transaction based on rules (which may be pre-defined or dynamically allocated) established by the permissionless blockchain **352** creators. For example, this may include verifying identities of the parties involved, etc. The transaction may be verified immediately or it may be placed in a queue with other transactions and the nodes **354** determine if the transactions are valid based on a set of network rules.

In structure **362**, valid transactions are formed into a block and sealed with a lock (hash). This process may be performed by mining nodes among the nodes **354**. Mining

nodes may utilize additional software specifically for mining and creating blocks for the permissionless blockchain 352. Each block may be identified by a hash (e.g., 256 bit number, etc.) created using an algorithm agreed upon by the network. Each block may include a header, a pointer or reference to a hash of a previous block's header in the chain, and a group of valid transactions. The reference to the previous block's hash is associated with the creation of the secure independent chain of blocks.

Before blocks can be added to the blockchain, the blocks must be validated. Validation for the permissionless blockchain 352 may include a proof-of-work (PoW) which is a solution to a puzzle derived from the block's header. Although not shown in the example of FIG. 3C, another process for validating a block is proof-of-stake. Unlike the proof-of-work, where the algorithm rewards miners who solve mathematical problems, with the proof of stake, a creator of a new block is chosen in a deterministic way, depending on its wealth, also defined as "stake." Then, a similar proof is performed by the selected/chosen node.

With mining 364, nodes try to solve the block by making incremental changes to one variable until the solution satisfies a network-wide target. This creates the PoW thereby ensuring correct answers. In other words, a potential solution must prove that computing resources were drained in solving the problem. In some types of permissionless blockchains, miners may be rewarded with value (e.g., coins, etc.) for correctly mining a block.

Here, the PoW process, alongside the chaining of blocks, makes modifications of the blockchain extremely difficult, as an attacker must modify all subsequent blocks in order for the modifications of one block to be accepted. Furthermore, as new blocks are mined, the difficulty of modifying a block is increased, and the number of subsequent blocks increases. With distribution 366, the successfully validated block is distributed through the permissionless blockchain 352 and all nodes 354 add the block to a majority chain which is the permissionless blockchain's 352 auditable ledger. Furthermore, the value in the transaction submitted by the sender 356 is deposited or otherwise transferred to the digital wallet of the recipient device 358.

The example embodiments may include various steps that are performed by entities involved in the identity-based encryption scheme. In the example embodiments, a transferor transfers an asset to a transferee (receiver) via a blockchain. However, contrary to a traditional blockchain network, in the example embodiments, the transaction can be executed on the blockchain prior to the transferee being onboarded to the blockchain. This process can be very helpful for situations where the buyer is not yet a member of the blockchain, such as a real-estate purchase, or the like. To perform the transfer, the blockchain network may create a temporary blockchain address to hold (and technically own) the asset until the transferee is successfully onboarded to the blockchain.

FIGS. 4A-4C illustrate a process of an identity-based encryption (IBE) scheme in which a transferor 410 transfers an asset to a blockchain 430 prior to a receiver 440 joining the blockchain 430 according to example embodiments, and FIG. 4D illustrates a process of the receiver 440 decrypting and transferring the asset to their own public key after the receiver 440 has joined the blockchain 430 according to example embodiments. In this example, the transferor 410 and the receiver 440 may refer to user devices in which users interact with blockchain software to transact on the blockchain 430. The user devices may be mobile devices, personal computers, servers, smart devices, or the like.

In the example IBE process described herein, a user identifier (e.g., an email address, a phone number, a passport ID, a social security number, etc.) of a transferee (e.g., the receiver 440 shown in FIG. 4D) can be used to randomly select "custodians" from the blockchain network to perform the transfer. In FIGS. 4A-4D the custodians are a subset of blockchain peers (i.e., peers 432, 434, and 436) which are selected as the custodians of the transfer. Here, the user identifier can be hashed by the transferor 410 to create a string value of a predetermined size. The string value can then be "sliced" starting on one end and working towards the other end of the string. It is likely that the process will never get to the end of the string as small sub-sections of the string (sub-strings) can be used as identifiers of blockchain peers who are to be custodians. As an example, the identity based encryption scheme as described herein may include the following algorithms.

Setup (λ) $\rightarrow$ (PP, mk)

An algorithm that receives as input a security parameter λ and outputs a global public key PP and a master private key mk.

Extract (mk, ID) $\rightarrow$ d_{ID}

An algorithm which receives as input a master private key mk, an identifier (e.g., an email) ID and outputs a private key d_{ID} that corresponds to the identifier and it is impossible to deduce the master private key mk from d_{ID} .

Encrypt (PP, ID, m) $\rightarrow$ c

An algorithm which receives as input a master private key mk, an identifier (e.g., an email) ID and outputs a private key d_{ID} that corresponds to the identifier and it is impossible to deduce the master private key mk from d_{ID} .

Decrypt (d_{ID} , c) $\rightarrow$ m

An algorithm which receives as input a global public key PP, an identifier ID, a message m, and outputs a ciphertext c.

An algorithm which receives as input a private key d_{ID} that corresponds to an identity ID, a ciphertext c, and outputs the ciphertext's decryption m.

During a setup step, public parameters PP (e.g., global public key of a custodian, etc.) and mk (e.g., a master private key of the custodian, etc.) can be generated. Each custodian (e.g., blockchain peers 432, 434, and 436) may perform the setup process and create their own key pairs. The setup process may be performed prior to the identity-based encryption process such as when the peers join/register with the blockchain 430.

When the transferor 410 desires to transfer an asset on the blockchain 430 to a transferee (receiver) who is not yet onboarded to the blockchain 430, the transferor may initiate an IBE process according to various embodiments. Here, the transferor of the asset may randomly select a subset of peers from the blockchain network as the "custodians" of the transfer of the asset. The transferor then provides a user identifier (e.g., an email address) of the transferee of the asset. In this step, each of the custodians execute the Extract algorithm which receives a respective master key of the custodian and the user identifier as inputs, and outputs a private key (d_{ID}).

For example, each of the custodians (e.g., the blockchain peers 432, 434, and 436) may generate a private and public key pair, execute an algorithm which receives a respective all public keys of the custodians, the user identifier, and the private key and generates a ciphertext. The ciphertext is homomorphic in that it can be operated on without being decrypted. The ciphertexts and public keys generated by the

custodians are obtained by the transferor. In response, the transferor combines the ciphertext into a single ciphertext and also generates a single public key from the plurality of public keys generated by the custodians. Next, the transferor stores the ciphertext at a network address of the blockchain that corresponds to the single public key. This transfers ownership of the asset to an address on the blockchain for temporary holding until the transferee can be onboarded and verified.

In the example embodiments, the transferee (e.g., the receiver **440** of the asset in the blockchain transaction) decrypts the ciphertext based on a secret key that can be obtained from the custodians. Here, the transferee must onboard to the blockchain and register a public/private key pair with the blockchain where the public key is used to identify a blockchain address of the transferee. Next, the transferee may perform the same hashing process on the user identifier to reveal the string value, and then use the same pseudo-random algorithm as the transferor used to identify the custodians from the string value. The transferee may then provide the user identifier (e.g., transferee's email address, etc.) to the custodians. In response, the custodians provide the transferee with secret keys of the custodian which can be combined to reveal a secret key that can decrypt the ciphertext *c* stored at the temporary blockchain address. Accordingly, the transferee can decrypt a combined private key within the ciphertext based on an algorithm which receives the private keys (dips) from the custodians and the ciphertext and outputs the message. Furthermore, the transferee may execute a blockchain transaction to transfer the asset from the temporary address to its own onboarded blockchain address.

As an example, the transfer process may include the following steps:

- 1) Issuer/Transferor (e.g., the transferor **410**) obtains email address of a party who is not yet onboarded to the blockchain **430** (e.g., the receiver **440**).
- 2) Issuer/transferor hashed the email address and users an algorithm on the hashed email address to select *f*+1 custodians (where it is assumed that less than *f* custodians are assumed to be dishonest).
- 3) Issuer/transferor sends the custodians the email address of the party. In response, each custodian (i) generates an Elliptic Curve key pair $\langle sk_i, pk_i \rangle$, encrypts the private key (ski) and sends back the public key (pki), the ciphertext *ci* which is

$$c_i = \text{Encrypt}(\sum_{i=1}^{f+1} PP_i, \text{email}, sk_i).$$

- 4) Issuer/transferor appends to the blockchain a combined ciphertext $\{c_i\}_{i=1}^{f+1}$ and adds all the public keys of the custodians into a single public key (pk) (e.g., $pk = pk_1 + pk_2 + \dots + pk_{f+1}$), and sets the owner to be the identifier of a credential with the public key (pk).

After the transferor appends the combined ciphertext to the blockchain network at the temporary blockchain address (public key), the temporary blockchain address essentially becomes the temporary owner of the asset until the asset is subsequently claimed by the recipient (e.g., the receiver **440**). The claiming process may include the following five steps:

- 1) Party (e.g., the receiver **440**) generates a private/public key pair.
- 2) Party onboards the blockchain (e.g., the blockchain **430**), and receives a credential identified by the public key.
- 3) Party hashes its email address and uses the same algorithm that the transferor **410** used on the hashed

email address to select *f*+1 custodians and the party then sends them the email address.

$$d_{ID} = \sum_{i=1}^{f+1} \text{Decrypt}(\sum_{i=1}^{f+1} d_{ID}^i, c_i)$$

FIG. 4A-4C illustrate a process of an identity-based encryption (IBE) scheme according to example embodiments. In the example embodiments, an IBE encryption scheme is used to create a temporary holding address (e.g., network address) of a blockchain ledger **430** for temporary custody of the transfer of the asset to a party that is not onboarded to the blockchain. When the party subsequently onboards itself to the blockchain, it can recover the temporary holding address based on the user identifier used during the IBE encryption scheme. Then, the party can transfer the asset from the temporary holding address to its own registered/onboarded blockchain address thereby completing the transfer.

Referring to FIG. 4A, a process **400A** of randomly selecting a subset of blockchain peers **432**, **434**, and **436** to be custodians is provided. Referring to FIG. 4A, a transferor **410** may generate a transfer transaction for transferring an asset to a transferee **440** (shown in FIG. 4D) via a blockchain ledger **430**. Here, the transferor **410** may have a user identifier of the transferee **440** such as an email address. The transferor **410** may hash the user identifier **412** to generate a hash value **414**, such as a binary string.

Next, the transferor **410** may slice the hash value **414** into smaller subsets **414-1**, **414-2**, **414-3**, . . . **414-N**. The subsets may be obtained by slicing off a predetermined number of digits/characters from the string beginning with a first side and working towards a second side of the string. Thus, each subset/slice may be non-overlapping with respect to the other subsets. Next, the transferor **410** may map the subsets **414-1**, **414-2**, and **414-3** to peer IDs of the blockchain peers **432**, **434**, and **436** via a mapping table. Any desired algorithm can be used such as assigning a range of binary values in the mapping table to each peer, etc. In this case, the subset can be mapped to a corresponding range in the mapping table and mapped to a corresponding peer assigned to the range. The randomness may be pseudo-random because it can be based off of a predefined algorithm that is known and an input that can be obtained from the blockchain ledger **430**. Therefore, any participant of the blockchain can verify that the correct custodians were selected by the transferor.

FIG. 4B illustrates a process **400B** of projecting the user identifier **412** onto the subset of blockchain peers **432**, **434**, and **436** which are randomly selected as the custodians. The transferor **410** may transmit the user identifier to each of the blockchain peers **434**, **434**, and **436**. In response, each of the blockchain peers **432**, **434**, and **436** may generate an elliptical curve key pair including a private key and a public key.

FIG. 4C illustrates a process of storing a ciphertext of the transfer transaction at a temporary address of the blockchain ledger **430**. Here, the blockchain peers **432**, **434**, and **436** create a respective ciphertext *c* by inputting a public key of the peer, the user identifier, and a message into an algorithm which creates a ciphertext *c*. Here, the transferor **410** may combine the ciphertexts from the blockchain peers **432**, **434**, and **436**, to generate a single combined ciphertext. In addition, the transferor **410** may combine the public keys (from the elliptical curve key pair) of the blockchain peers **432**, **434**, and **436**, to generate a single public key that can be used to identify a temporary blockchain address of the blockchain ledger **430**. Next, the ciphertext with the encrypted asset transfer content may be stored at the temporary blockchain address until the transferee **440** is onboarded.

15

FIG. 4D illustrates a process 400D of the transferee 440 claiming the asset according to example embodiments. Referring to FIG. 4D, the transferee 440 can hash the same user identifier 412 and generate the string value. Furthermore, the transferee 440 can use the same algorithm as used by the transferor 410 to identify the subset of peer nodes 432, 434, and 436 from a larger pool of nodes on the blockchain network. Then, the transferee 440 can send the user identifier 412 to the blockchain peers 432, 434, and 436. In response, the blockchain peers 432, 434, and 436 can return the private keys (from the elliptical curve key pair) which can be combined by the transferee 440 to recover a secret key that can decrypt the ciphertext with the encrypted asset details. Next, the transferee 440 can execute a blockchain transaction to transfer the asset to the transferee's public address from the onboarding process.

FIG. 5 illustrates a method 500 of an identity-based encryption scheme for a future transfer of an asset according to example embodiments. As a non-limiting example, the method 500 may be performed by a computing device such as a user device, a blockchain computer/server, a virtual machine, a web server, a cloud platform, and the like. Referring to FIG. 5, in 510, the method may include transmitting a user identifier to a plurality of blockchain peers of a blockchain network. For example, the user identifier may be an email address, a phone number, a social security number, a driver's license number, a passport ID, or any other user identifier. The user identifier may represent a user who is not yet onboarded to a blockchain but who is the recipient of the asset.

In 520, the method may include receiving a plurality of public keys from the plurality of blockchain peers. In 530, the method may include receiving a plurality of ciphertexts from the plurality of blockchain peers generated based on the user identifier and a corresponding plurality of private keys of the plurality of blockchain peers. In 540, the method may include generating a combined ciphertext that includes the plurality of ciphertexts and generating a temporary blockchain address of the blockchain network based on the plurality of public keys. In 550, the method may include storing the combined ciphertext and a transfer of an asset via the temporary blockchain address of the blockchain network.

When the recipient is subsequently onboarded to the blockchain, the recipient may use a public key that they have obtained to register with the blockchain. The public key may be used as a credential for a blockchain address of the blockchain network. Accordingly, in 560, the recipient receive the plurality of private keys (dips) from the custodians as described in the process of FIG. 4D, and decrypt the ciphertext and output the private key controlling the temporary address of the asset. Furthermore, the transferee may execute a blockchain transaction to transfer the asset from the temporary address to its own onboarded blockchain address.

In some embodiments, the method may further include randomly selecting the plurality of blockchain peers based on a hash of the user identifier and a pseudo-random algorithm. In some embodiments, the generating the temporary blockchain address may include adding together the plurality of public keys to create a single public key, and storing to blockchain transaction with the transfer of the asset via a blockchain address corresponding to the single public key. In some embodiments, the randomly selecting may include hashing the user identifier into a string value, slicing off a plurality of sub-strings from the string value,

16

and mapping the plurality of sliced sub-strings to a plurality of respective identifiers of the plurality of blockchain peers in a table.

In some embodiments, the receiving may include receiving a plurality of public keys from a plurality of elliptical curve key pairs generated by the plurality of blockchain peers, where the plurality of ciphertexts are encrypted with a plurality of private keys from the plurality of elliptical curve key pairs. In some embodiments, the plurality of ciphertexts are added together via a homomorphic process. In some embodiments, the user identifier may include an email address of a receiver of the transfer of the asset, and the receiver is not a registered participant of the blockchain network.

FIG. 6A illustrates an example system 600 that includes a physical infrastructure 610 configured to perform various operations according to example embodiments. Referring to FIG. 6A, the physical infrastructure 610 includes a module 612 and a module 614. The module 614 includes a blockchain 620 and a smart contract 630 (which may reside on the blockchain 620), that may execute any of the operational steps 608 (in module 612) included in any of the example embodiments. The steps/operations 608 may include one or more of the embodiments described or depicted and may represent output or written information that is written or read from one or more smart contracts 630 and/or blockchains 620. The physical infrastructure 610, the module 612, and the module 614 may include one or more computers, servers, processors, memories, and/or wireless communication devices. Further, the module 612 and the module 614 may be a same module.

FIG. 6B illustrates another example system 640 configured to perform various operations according to example embodiments. Referring to FIG. 6B, the system 640 includes a module 612 and a module 614. The module 614 includes a blockchain 620 and a smart contract 630 (which may reside on the blockchain 620), that may execute any of the operational steps 608 (in module 612) included in any of the example embodiments. The steps/operations 608 may include one or more of the embodiments described or depicted and may represent output or written information that is written or read from one or more smart contracts 630 and/or blockchains 620. The physical infrastructure 610, the module 612, and the module 614 may include one or more computers, servers, processors, memories, and/or wireless communication devices. Further, the module 612 and the module 614 may be a same module.

FIG. 6C illustrates an example system configured to utilize a smart contract configuration among contracting parties and a mediating server configured to enforce the smart contract terms on the blockchain according to example embodiments. Referring to FIG. 6C, the configuration 650 may represent a communication session, an asset transfer session or a process or procedure that is driven by a smart contract 630 which explicitly identifies one or more user devices 652 and/or 656. The execution, operations and results of the smart contract execution may be managed by a server 654. Content of the smart contract 630 may require digital signatures by one or more of the entities 652 and 656 which are parties to the smart contract transaction. The results of the smart contract execution may be written to a blockchain 620 as a blockchain transaction. The smart contract 630 resides on the blockchain 620 which may reside on one or more computers, servers, processors, memories, and/or wireless communication devices.

FIG. 6D illustrates a system 660 including a blockchain, according to example embodiments. Referring to the

example of FIG. 6D, an application programming interface (API) gateway **662** provides a common interface for accessing blockchain logic (e.g., smart contract **630** or other chaincode) and data (e.g., distributed ledger, etc.). In this example, the API gateway **662** is a common interface for performing transactions (invoke, queries, etc.) on the blockchain by connecting one or more entities **652** and **656** to a blockchain peer (i.e., server **654**). Here, the server **654** is a blockchain network peer component that holds a copy of the world state and a distributed ledger allowing clients **652** and **656** to query data on the world state as well as submit transactions into the blockchain network where, depending on the smart contract **630** and endorsement policy, endorsing peers will run the smart contracts **630**.

The above embodiments may be implemented in hardware, in a computer program executed by a processor, in firmware, or in a combination of the above. A computer program may be embodied on a computer readable medium, such as a storage medium. For example, a computer program may reside in random access memory ("RAM"), flash memory, read-only memory ("ROM"), erasable programmable read-only memory ("EPROM"), electrically erasable programmable read-only memory ("EEPROM"), registers, hard disk, a removable disk, a compact disk read-only memory ("CD-ROM"), or any other form of storage medium known in the art.

An exemplary storage medium may be coupled to the processor such that the processor may read information from, and write information to, the storage medium. In the alternative, the storage medium may be integral to the processor. The processor and the storage medium may reside in an application specific integrated circuit ("ASIC"). In the alternative, the processor and the storage medium may reside as discrete components.

FIG. 7A illustrates a process **700** of a new block being added to a distributed ledger **720**, according to example embodiments, and FIG. 7B illustrates contents of a new data block structure **730** for blockchain, according to example embodiments. Referring to FIG. 7A, clients (not shown) may submit transactions to blockchain nodes **711**, **712**, and/or **713**. Clients may be instructions received from any source to enact activity on the blockchain **720**. As an example, clients may be applications that act on behalf of a requester, such as a device, person or entity to propose transactions for the blockchain. The plurality of blockchain peers (e.g., blockchain nodes **711**, **712**, and **713**) may maintain a state of the blockchain network and a copy of the distributed ledger **720**. Different types of blockchain nodes/peers may be present in the blockchain network including endorsing peers which simulate and endorse transactions proposed by clients and committing peers which verify endorsements, validate transactions, and commit transactions to the distributed ledger **720**. In this example, the blockchain nodes **711**, **712**, and **713** may perform the role of endorser node, committer node, or both.

The distributed ledger **720** includes a blockchain which stores immutable, sequenced records in blocks, and a state database **724** (current world state) maintaining a current state of the blockchain **722**. One distributed ledger **720** may exist per channel and each peer maintains its own copy of the distributed ledger **720** for each channel of which they are a member. The blockchain **722** is a transaction log, structured as hash-linked blocks where each block contains a sequence of N transactions. Blocks may include various components such as shown in FIG. 7B. The linking of the blocks (shown by arrows in FIG. 7A) may be generated by adding a hash of a prior block's header within a block header

of a current block. In this way, all transactions on the blockchain **722** are sequenced and cryptographically linked together preventing tampering with blockchain data without breaking the hash links. Furthermore, because of the links, the latest block in the blockchain **722** represents every transaction that has come before it. The blockchain **722** may be stored on a peer file system (local or attached storage), which supports an append-only blockchain workload.

The current state of the blockchain **722** and the distributed ledger **722** may be stored in the state database **724**. Here, the current state data represents the latest values for all keys ever included in the chain transaction log of the blockchain **722**. Chaincode invocations execute transactions against the current state in the state database **724**. To make these chaincode interactions extremely efficient, the latest values of all keys are stored in the state database **724**. The state database **724** may include an indexed view into the transaction log of the blockchain **722**, it can therefore be regenerated from the chain at any time. The state database **724** may automatically get recovered (or generated if needed) upon peer startup, before transactions are accepted.

Endorsing nodes receive transactions from clients and endorse the transaction based on simulated results. Endorsing nodes hold smart contracts which simulate the transaction proposals. When an endorsing node endorses a transaction, the endorsing nodes creates a transaction endorsement which is a signed response from the endorsing node to the client application indicating the endorsement of the simulated transaction. The method of endorsing a transaction depends on an endorsement policy which may be specified within chaincode. An example of an endorsement policy is "the majority of endorsing peers must endorse the transaction". Different channels may have different endorsement policies. Endorsed transactions are forward by the client application to ordering service **710**.

The ordering service **710** accepts endorsed transactions, orders them into a block, and delivers the blocks to the committing peers. For example, the ordering service **710** may initiate a new block when a threshold of transactions has been reached, a timer times out, or another condition. In the example of FIG. 7A, blockchain node **712** is a committing peer that has received a new data new data block **730** for storage on blockchain **720**. The first block in the blockchain may be referred to as a genesis block which includes information about the blockchain, its members, the data stored therein, etc.

The ordering service **710** may be made up of a cluster of orderers. The ordering service **710** does not process transactions, smart contracts, or maintain the shared ledger. Rather, the ordering service **710** may accept the endorsed transactions and specifies the order in which those transactions are committed to the distributed ledger **720**. The architecture of the blockchain network may be designed such that the specific implementation of 'ordering' (e.g., Solo, Kafka, BFT, etc.) becomes a pluggable component.

Transactions are written to the distributed ledger **720** in a consistent order. The order of transactions is established to ensure that the updates to the state database **724** are valid when they are committed to the network. Unlike a cryptocurrency blockchain system (e.g., Bitcoin, etc.) where ordering occurs through the solving of a cryptographic puzzle, or mining, in this example the parties of the distributed ledger **720** may choose the ordering mechanism that best suits that network.

When the ordering service **710** initializes a new data block **730**, the new data block **730** may be broadcast to committing peers (e.g., blockchain nodes **711**, **712**, and

713). In response, each committing peer validates the transaction within the new data block 730 by checking to make sure that the read set and the write set still match the current world state in the state database 724. Specifically, the committing peer can determine whether the read data that existed when the endorsers simulated the transaction is identical to the current world state in the state database 724. When the committing peer validates the transaction, the transaction is written to the blockchain 722 on the distributed ledger 720, and the state database 724 is updated with the write data from the read-write set. If a transaction fails, that is, if the committing peer finds that the read-write set does not match the current world state in the state database 724, the transaction ordered into a block will still be included in that block, but it will be marked as invalid, and the state database 724 will not be updated.

Referring to FIG. 7B, a new data block 730 (also referred to as a data block) that is stored on the blockchain 722 of the distributed ledger 720 may include multiple data segments such as a block header 740, block data 750 (block data section), and block metadata 760. It should be appreciated that the various depicted blocks and their contents, such as new data block 730 and its contents, shown in FIG. 7B are merely examples and are not meant to limit the scope of the example embodiments. In a conventional block, the data section may store transactional information of N transaction(s) (e.g., 1, 10, 100, 500, 1000, 2000, 3000, etc.) within the block data 750.

The new data block 730 may also include a link to a previous block (e.g., on the blockchain 722 in FIG. 7A) within the block header 740. In particular, the block header 740 may include a hash of a previous block's header. The block header 740 may also include a unique block number, a hash of the block data 750 of the new data block 730, and the like. The block number of the new data block 730 may be unique and assigned in various orders, such as an incremental/sequential order starting from zero.

According to various embodiments, the block data 750 may also store attributes 752 associated with a blockchain transaction which are consumed by the smart contract described herein. For example, the attributes 752 may include the global public key (PPI) for each custodian, the public key (pk) for an asset/transfer, and the ciphertext combined together by the transferor. The attributes 752 include one or more of the steps, features, processes and/or actions described or depicted herein. Accordingly, the attributes 752 can be stored in an immutable log of blocks on the distributed ledger 720. Some of the benefits of storing the attributes 752 on the blockchain are reflected in the various embodiments disclosed and depicted herein. Although in FIG. 7B, the attributes 752 are depicted in the block data 750, it may be located in the block header 740 or the block metadata 760.

The block metadata 760 may store multiple fields of metadata (e.g., as a byte array, etc.). Metadata fields may include signature on block creation, a reference to a last configuration block, a transaction filter identifying valid and invalid transactions within the block, last offset persisted of an ordering service that ordered the block, and the like. The signature, the last configuration block, and the orderer metadata may be added by the ordering service 710. Meanwhile, a committer of the block (such as blockchain node 712) may add validity/invalidity information based on an endorsement policy, verification of read/write sets, and the like. The transaction filter may include a byte array of a size

equal to the number of transactions that are included in the block data 750 and a validation code identifying whether a transaction was valid/invalid.

FIG. 7C illustrates an embodiment of a blockchain 770 for digital content in accordance with the embodiments described herein. The digital content may include one or more files and associated information. The files may include media, images, video, audio, text, links, graphics, animations, web pages, documents, or other forms of digital content. The immutable, append-only aspects of the blockchain serve as a safeguard to protect the integrity, validity, and authenticity of the digital content, making it suitable use in legal proceedings where admissibility rules apply or other settings where evidence is taken into consideration or where the presentation and use of digital information is otherwise of interest. In this case, the digital content may be referred to as digital evidence.

The blockchain may be formed in various ways. In one embodiment, the digital content may be included in and accessed from the blockchain itself. For example, each block of the blockchain may store a hash value of reference information (e.g., header, value, etc.) along the associated digital content. The hash value and associated digital content may then be encrypted together. Thus, the digital content of each block may be accessed by decrypting each block in the blockchain, and the hash value of each block may be used as a basis to reference a previous block. This may be illustrated as follows:

Block 1	Block 2	...	Block N
Hash Value 1	Hash Value 2		Hash Value N
Digital Content 1	Digital Content 2		Digital Content N

In one embodiment, the digital content may be not included in the blockchain. For example, the blockchain may store the encrypted hashes of the content of each block without any of the digital content. The digital content may be stored in another storage area or memory address in association with the hash value of the original file. The other storage area may be the same storage device used to store the blockchain or may be a different storage area or even a separate relational database. The digital content of each block may be referenced or accessed by obtaining or querying the hash value of a block of interest and then looking up that has value in the storage area, which is stored in correspondence with the actual digital content. This operation may be performed, for example, a database gatekeeper. This may be illustrated as follows:

Blockchain	Storage Area
Block 1 Hash Value	Block 1 Hash Value . . . Content
.	.
.	.
Block N Hash Value	Block N Hash Value . . . Content

In the example embodiment of FIG. 7C, the blockchain 770 includes a number of blocks 778₁, 778₂, . . . 778_N cryptographically linked in an ordered sequence, where N ≥ 1. The encryption used to link the blocks 778₁, 778₂, . . . 778_N may be any of a number of keyed or un-keyed Hash functions. In one embodiment, the blocks 778₁, 778₂, . . . 778_N are subject to a hash function which produces n-bit alphanumeric outputs (where n is 256 or another

number) from inputs that are based on information in the blocks. Examples of such a hash function include, but are not limited to, a SHA-type (SHA stands for Secured Hash Algorithm) algorithm, Merkle-Damgard algorithm, HAIFA algorithm, Merkle-tree algorithm, nonce-based algorithm, and a non-collision-resistant PRF algorithm. In another embodiment, the blocks $778_1, 778_2, \dots, 778_N$ may be cryptographically linked by a function that is different from a hash function. For purposes of illustration, the following description is made with reference to a hash function, e.g., SHA-2.

Each of the blocks $778_1, 778_2, \dots, 778_N$ in the blockchain includes a header, a version of the file, and a value. The header and the value are different for each block as a result of hashing in the blockchain. In one embodiment, the value may be included in the header. As described in greater detail below, the version of the file may be the original file or a different version of the original file.

The first block 778_1 in the blockchain is referred to as the genesis block and includes the header 772_1 , original file 774_1 , and an initial value 776_1 . The hashing scheme used for the genesis block, and indeed in all subsequent blocks, may vary. For example, all the information in the first block 778_1 may be hashed together and at one time, or each or a portion of the information in the first block 778_1 may be separately hashed and then a hash of the separately hashed portions may be performed.

The header 772_1 may include one or more initial parameters, which, for example, may include a version number, timestamp, nonce, root information, difficulty level, consensus protocol, duration, media format, source, descriptive keywords, and/or other information associated with original file 774_1 and/or the blockchain. The header 772_1 may be generated automatically (e.g., by blockchain network managing software) or manually by a blockchain participant. Unlike the header in other blocks 778_2 to 778_N in the blockchain, the header 772_1 in the genesis block does not reference a previous block, simply because there is no previous block.

The original file 774_1 in the genesis block may be, for example, data as captured by a device with or without processing prior to its inclusion in the blockchain. The original file 774_1 is received through the interface of the system from the device, media source, or node. The original file 774_1 is associated with metadata, which, for example, may be generated by a user, the device, and/or the system processor, either manually or automatically. The metadata may be included in the first block 778_1 in association with the original file 774_1 .

The value 776_1 in the genesis block is an initial value generated based on one or more unique attributes of the original file 774_1 . In one embodiment, the one or more unique attributes may include the hash value for the original file 774_1 , metadata for the original file 774_1 , and other information associated with the file. In one implementation, the initial value 776_1 may be based on the following unique attributes:

- 1) SHA-2 computed hash value for the original file
- 2) originating device ID
- 3) starting timestamp for the original file
- 4) initial storage location of the original file
- 5) blockchain network member ID for software to currently control the original file and associated metadata

The other blocks 778_2 to 778_N in the blockchain also have headers, files, and values. However, unlike the first block 778_1 , each of the headers 772_2 to 772_N in the other blocks includes the hash value of an immediately preceding block.

The hash value of the immediately preceding block may be just the hash of the header of the previous block or may be the hash value of the entire previous block. By including the hash value of a preceding block in each of the remaining blocks, a trace can be performed from the Nth block back to the genesis block (and the associated original file) on a block-by-block basis, as indicated by arrows 780 , to establish an auditable and immutable chain-of-custody.

Each of the header 772_2 to 772_N in the other blocks may also include other information, e.g., version number, timestamp, nonce, root information, difficulty level, consensus protocol, and/or other parameters or information associated with the corresponding files and/or the blockchain in general.

The files 774_2 to 774_N in the other blocks may be equal to the original file or may be a modified version of the original file in the genesis block depending, for example, on the type of processing performed. The type of processing performed may vary from block to block. The processing may involve, for example, any modification of a file in a preceding block, such as redacting information or otherwise changing the content of, taking information away from, or adding or appending information to the files.

Additionally, or alternatively, the processing may involve merely copying the file from a preceding block, changing a storage location of the file, analyzing the file from one or more preceding blocks, moving the file from one storage or memory location to another, or performing action relative to the file of the blockchain and/or its associated metadata. Processing which involves analyzing a file may include, for example, appending, including, or otherwise associating various analytics, statistics, or other information associated with the file.

The values in each of the other blocks 776_2 to 776_N in the other blocks are unique values and are all different as a result of the processing performed. For example, the value in any one block corresponds to an updated version of the value in the previous block. The update is reflected in the hash of the block to which the value is assigned. The values of the blocks therefore provide an indication of what processing was performed in the blocks and also permit a tracing through the blockchain back to the original file. This tracking confirms the chain-of-custody of the file throughout the entire blockchain.

For example, consider the case where portions of the file in a previous block are redacted, blocked out, or pixelated in order to protect the identity of a person shown in the file. In this case, the block including the redacted file will include metadata associated with the redacted file, e.g., how the redaction was performed, who performed the redaction, timestamps where the redaction(s) occurred, etc. The metadata may be hashed to form the value. Because the metadata for the block is different from the information that was hashed to form the value in the previous block, the values are different from one another and may be recovered when decrypted.

In one embodiment, the value of a previous block may be updated (e.g., a new hash value computed) to form the value of a current block when any one or more of the following occurs. The new hash value may be computed by hashing all or a portion of the information noted below, in this example embodiment.

- a) new SHA-2 computed hash value if the file has been processed in any way (e.g., if the file was redacted, copied, altered, accessed, or some other action was taken)
- b) new storage location for the file

23

- c) new metadata identified associated with the file
- d) transfer of access or control of the file from one blockchain participant to another blockchain participant

FIG. 7D illustrates an embodiment of a block which may represent the structure of the blocks in the blockchain 790 in accordance with one embodiment. The block, Block_i, includes a header 772_i, a file 774_i, and a value 776_i.

The header 772_i includes a hash value of a previous block Block_{i-1} and additional reference information, which, for example, may be any of the types of information (e.g., header information including references, characteristics, parameters, etc.) discussed herein. All blocks reference the hash of a previous block except, of course, the genesis block. The hash value of the previous block may be just a hash of the header in the previous block or a hash of all or a portion of the information in the previous block, including the file and metadata.

The file 774_i includes a plurality of data, such as Data 1, Data 2, . . . , Data N in sequence. The data are tagged with metadata Metadata 1, Metadata 2, . . . , Metadata N which describe the content and/or characteristics associated with the data. For example, the metadata for each data may include information to indicate a timestamp for the data, process the data, keywords indicating the persons or other content depicted in the data, and/or other features that may be helpful to establish the validity and content of the file as a whole, and particularly its use a digital evidence, for example, as described in connection with an embodiment discussed below. In addition to the metadata, each data may be tagged with reference REF₁, REF₂, . . . , REF_N to a previous data to prevent tampering, gaps in the file, and sequential reference through the file.

Once the metadata is assigned to the data (e.g., through a smart contract), the metadata cannot be altered without the hash changing, which can easily be identified for invalidation. The metadata, thus, creates a data log of information that may be accessed for use by participants in the blockchain.

The value 776_i is a hash value or other value computed based on any of the types of information previously discussed. For example, for any given block Block_i, the value for that block may be updated to reflect the processing that was performed for that block, e.g., new hash value, new storage location, new metadata for the associated file, transfer of control or access, identifier, or other action or information to be added. Although the value in each block is shown to be separate from the metadata for the data of the file and header, the value may be based, in part or whole, on this metadata in another embodiment.

Once the blockchain 770 is formed, at any point in time, the immutable chain-of-custody for the file may be obtained by querying the blockchain for the transaction history of the values across the blocks. This query, or tracking procedure, may begin with decrypting the value of the block that is most currently included (e.g., the last (Nth) block), and then continuing to decrypt the value of the other blocks until the genesis block is reached and the original file is recovered. The decryption may involve decrypting the headers and files and associated metadata at each block, as well.

Decryption is performed based on the type of encryption that took place in each block. This may involve the use of private keys, public keys, or a public key-private key pair. For example, when asymmetric encryption is used, blockchain participants or a processor in the network may generate a public key and private key pair using a predetermined algorithm. The public key and private key are associated

24

with each other through some mathematical relationship. The public key may be distributed publicly to serve as an address to receive messages from other users, e.g., an IP address or home address. The private key is kept secret and used to digitally sign messages sent to other blockchain participants. The signature is included in the message so that the recipient can verify using the public key of the sender. This way, the recipient can be sure that only the sender could have sent this message.

Generating a key pair may be analogous to creating an account on the blockchain, but without having to actually register anywhere. Also, every transaction that is executed on the blockchain is digitally signed by the sender using their private key. This signature ensures that only the owner of the account can track and process (if within the scope of permission determined by a smart contract) the file of the blockchain.

FIGS. 8A and 8B illustrate additional examples of use cases for blockchain which may be incorporated and used herein. In particular, FIG. 8A illustrates an example 800 of a blockchain 810 which stores machine learning (artificial intelligence) data. Machine learning relies on vast quantities of historical data (or training data) to build predictive models for accurate prediction on new data. Machine learning software (e.g., neural networks, etc.) can often sift through millions of records to unearth non-intuitive patterns.

In the example of FIG. 8A, a host platform 820 builds and deploys a machine learning model for predictive monitoring of assets 830. Here, the host platform 820 may be a cloud platform, an industrial server, a web server, a personal computer, a user device, and the like. Assets 830 can be any type of asset (e.g., machine or equipment, etc.) such as an aircraft, locomotive, turbine, medical machinery and equipment, oil and gas equipment, boats, ships, vehicles, and the like. As another example, assets 830 may be non-tangible assets such as stocks, currency, digital coins, insurance, or the like.

The blockchain 810 can be used to significantly improve both a training process 802 of the machine learning model and a predictive process 804 based on a trained machine learning model. For example, in 802, rather than requiring a data scientist/engineer or other user to collect the data, historical data may be stored by the assets 830 themselves (or through an intermediary, not shown) on the blockchain 810. This can significantly reduce the collection time needed by the host platform 820 when performing predictive model training. For example, using smart contracts, data can be directly and reliably transferred straight from its place of origin to the blockchain 810. By using the blockchain 810 to ensure the security and ownership of the collected data, smart contracts may directly send the data from the assets to the individuals that use the data for building a machine learning model. This allows for sharing of data among the assets 830.

The collected data may be stored in the blockchain 810 based on a consensus mechanism. The consensus mechanism pulls in (permissioned nodes) to ensure that the data being recorded is verified and accurate. The data recorded is time-stamped, cryptographically signed, and immutable. It is therefore auditable, transparent, and secure. Adding IoT devices which write directly to the blockchain can, in certain cases (i.e. supply chain, healthcare, logistics, etc.), increase both the frequency and accuracy of the data being recorded.

Furthermore, training of the machine learning model on the collected data may take rounds of refinement and testing by the host platform 820. Each round may be based on additional data or data that was not previously considered to

help expand the knowledge of the machine learning model. In **802**, the different training and testing steps (and the data associated therewith) may be stored on the blockchain **810** by the host platform **820**. Each refinement of the machine learning model (e.g., changes in variables, weights, etc.) may be stored on the blockchain **810**. This provides verifiable proof of how the model was trained and what data was used to train the model. Furthermore, when the host platform **820** has achieved a finally trained model, the resulting model may be stored on the blockchain **810**.

After the model has been trained, it may be deployed to a live environment where it can make predictions/decisions based on the execution of the final trained machine learning model. For example, in **804**, the machine learning model may be used for condition-based maintenance (CBM) for an asset such as an aircraft, a wind turbine, a healthcare machine, and the like. In this example, data fed back from the asset **830** may be input the machine learning model and used to make event predictions such as failure events, error codes, and the like. Determinations made by the execution of the machine learning model at the host platform **820** may be stored on the blockchain **810** to provide auditable/verifiable proof. As one non-limiting example, the machine learning model may predict a future breakdown/failure to a part of the asset **830** and create alert or a notification to replace the part. The data behind this decision may be stored by the host platform **820** on the blockchain **810**. In one embodiment the features and/or the actions described and/or depicted herein can occur on or with respect to the blockchain **810**.

New transactions for a blockchain can be gathered together into a new block and added to an existing hash value. This is then encrypted to create a new hash for the new block. This is added to the next list of transactions when they are encrypted, and so on. The result is a chain of blocks that each contain the hash values of all preceding blocks. Computers that store these blocks regularly compare their hash values to ensure that they are all in agreement. Any computer that does not agree, discards the records that are causing the problem. This approach is good for ensuring tamper-resistance of the blockchain, but it is not perfect.

One way to game this system is for a dishonest user to change the list of transactions in their favor, but in a way that leaves the hash unchanged. This can be done by brute force, in other words by changing a record, encrypting the result, and seeing whether the hash value is the same. And if not, trying again and again and again until it finds a hash that matches. The security of blockchains is based on the belief that ordinary computers can only perform this kind of brute force attack over time scales that are entirely impractical, such as the age of the universe. By contrast, quantum computers are much faster (1000 s of times faster) and consequently pose a much greater threat.

FIG. **8B** illustrates an example **850** of a quantum-secure blockchain **852** which implements quantum key distribution (QKD) to protect against a quantum computing attack. In this example, blockchain users can verify each other's identities using QKD. This sends information using quantum particles such as photons, which cannot be copied by an eavesdropper without destroying them. In this way, a sender and a receiver through the blockchain can be sure of each other's identity.

In the example of FIG. **8B**, four users are present **854**, **856**, **858**, and **860**. Each of pair of users may share a secret key **862** (i.e., a QKD) between themselves. Since there are four nodes in this example, six pairs of nodes exists, and therefore six different secret keys **862** are used including

QKD_{AB} , QKD_{AC} , QKD_{AD} , QKD_{BC} , QKD_{BD} , and QKD_{CD} . Each pair can create a QKD by sending information using quantum particles such as photons, which cannot be copied by an eavesdropper without destroying them. In this way, a pair of users can be sure of each other's identity.

The operation of the blockchain **852** is based on two procedures (i) creation of transactions, and (ii) construction of blocks that aggregate the new transactions. New transactions may be created similar to a traditional blockchain network. Each transaction may contain information about a sender, a receiver, a time of creation, an amount (or value) to be transferred, a list of reference transactions that justifies the sender has funds for the operation, and the like. This transaction record is then sent to all other nodes where it is entered into a pool of unconfirmed transactions. Here, two parties (i.e., a pair of users from among **854-860**) authenticate the transaction by providing their shared secret key **862** (QKD). This quantum signature can be attached to every transaction making it exceedingly difficult to tamper with. Each node checks their entries with respect to a local copy of the blockchain **852** to verify that each transaction has sufficient funds. However, the transactions are not yet confirmed.

Rather than perform a traditional mining process on the blocks, the blocks may be created in a decentralized manner using a broadcast protocol. At a predetermined period of time (e.g., seconds, minutes, hours, etc.) the network may apply the broadcast protocol to any unconfirmed transaction thereby to achieve a Byzantine agreement (consensus) regarding a correct version of the transaction. For example, each node may possess a private value (transaction data of that particular node). In a first round, nodes transmit their private values to each other. In subsequent rounds, nodes communicate the information they received in the previous round from other nodes. Here, honest nodes are able to create a complete set of transactions within a new block. This new block can be added to the blockchain **852**. In one embodiment the features and/or the actions described and/or depicted herein can occur on or with respect to the blockchain **852**.

FIG. **9** illustrates an example system **900** that supports one or more of the example embodiments described and/or depicted herein. The system **900** comprises a computer system/server **902**, which is operational with numerous other general purpose or special purpose computing system environments or configurations. Examples of well-known computing systems, environments, and/or configurations that may be suitable for use with computer system/server **902** include, but are not limited to, personal computer systems, server computer systems, thin clients, thick clients, hand-held or laptop devices, multiprocessor systems, microprocessor-based systems, set top boxes, programmable consumer electronics, network PCs, minicomputer systems, mainframe computer systems, and distributed cloud computing environments that include any of the above systems or devices, and the like.

Computer system/server **902** may be described in the general context of computer system-executable instructions, such as program modules, being executed by a computer system. Generally, program modules may include routines, programs, objects, components, logic, data structures, and so on that perform particular tasks or implement particular abstract data types. Computer system/server **902** may be practiced in distributed cloud computing environments where tasks are performed by remote processing devices that are linked through a communications network. In a distributed cloud computing environment, program modules may

be located in both local and remote computer system storage media including memory storage devices.

As shown in FIG. 9, computer system/server 902 in cloud computing node 900 is shown in the form of a general-purpose computing device. The components of computer system/server 902 may include, but are not limited to, one or more processors or processing units 904, a system memory 906, and a bus that couples various system components including system memory 906 to processor 904.

The bus represents one or more of any of several types of bus structures, including a memory bus or memory controller, a peripheral bus, an accelerated graphics port, and a processor or local bus using any of a variety of bus architectures. By way of example, and not limitation, such architectures include Industry Standard Architecture (ISA) bus, Micro Channel Architecture (MCA) bus, Enhanced ISA (EISA) bus, Video Electronics Standards Association (VESA) local bus, and Peripheral Component Interconnects (PCI) bus.

Computer system/server 902 typically includes a variety of computer system readable media. Such media may be any available media that is accessible by computer system/server 902, and it includes both volatile and non-volatile media, removable and non-removable media. System memory 906, in one embodiment, implements the flow diagrams of the other figures. The system memory 906 can include computer system readable media in the form of volatile memory, such as random-access memory (RAM) 910 and/or cache memory 912. Computer system/server 902 may further include other removable/non-removable, volatile/non-volatile computer system storage media. By way of example only, storage system 914 can be provided for reading from and writing to a non-removable, non-volatile magnetic media (not shown and typically called a "hard drive"). Although not shown, a magnetic disk drive for reading from and writing to a removable, non-volatile magnetic disk (e.g., a "floppy disk"), and an optical disk drive for reading from or writing to a removable, non-volatile optical disk such as a CD-ROM, DVD-ROM or other optical media can be provided. In such instances, each can be connected to the bus by one or more data media interfaces. As will be further depicted and described below, memory 906 may include at least one program product having a set (e.g., at least one) of program modules that are configured to carry out the functions of various embodiments of the application.

Program/utility 916, having a set (at least one) of program modules 918, may be stored in memory 906 by way of example, and not limitation, as well as an operating system, one or more application programs, other program modules, and program data. Each of the operating system, one or more application programs, other program modules, and program data or some combination thereof, may include an implementation of a networking environment. Program modules 918 generally carry out the functions and/or methodologies of various embodiments of the application as described herein.

As will be appreciated by one skilled in the art, aspects of the present application may be embodied as a system, method, or computer program product. Accordingly, aspects of the present application may take the form of an entirely hardware embodiment, an entirely software embodiment (including firmware, resident software, micro-code, etc.) or an embodiment combining software and hardware aspects that may all generally be referred to herein as a "circuit," "module" or "system." Furthermore, aspects of the present application may take the form of a computer program

product embodied in one or more computer readable medium(s) having computer readable program code embodied thereon.

Computer system/server 902 may also communicate with one or more external devices 920 such as a keyboard, a pointing device, a display 922, etc.; one or more devices that enable a user to interact with computer system/server 902; and/or any devices (e.g., network card, modem, etc.) that enable computer system/server 902 to communicate with one or more other computing devices. Such communication can occur via I/O interfaces 924. Still yet, computer system/server 902 can communicate with one or more networks such as a local area network (LAN), a general wide area network (WAN), and/or a public network (e.g., the Internet) via network adapter 926. As depicted, network adapter 926 communicates with the other components of computer system/server 902 via a bus. It should be understood that although not shown, other hardware and/or software components could be used in conjunction with computer system/server 902. Examples, include, but are not limited to: microcode, device drivers, redundant processing units, external disk drive arrays, RAID systems, tape drives, and data archival storage systems, etc.

Although an exemplary embodiment of at least one of a system, method, and non-transitory computer readable medium has been illustrated in the accompanied drawings and described in the foregoing detailed description, it will be understood that the application is not limited to the embodiments disclosed, but is capable of numerous rearrangements, modifications, and substitutions as set forth and defined by the following claims. For example, the capabilities of the system of the various figures can be performed by one or more of the modules or components described herein or in a distributed architecture and may include a transmitter, receiver or pair of both. For example, all or part of the functionality performed by the individual modules, may be performed by one or more of these modules. Further, the functionality described herein may be performed at various times and in relation to various events, internal or external to the modules or components. Also, the information sent between various modules can be sent between the modules via at least one of: a data network, the Internet, a voice network, an Internet Protocol network, a wireless device, a wired device and/or via plurality of protocols. Also, the messages sent or received by any of the modules may be sent or received directly and/or via one or more of the other modules.

One skilled in the art will appreciate that a "system" could be embodied as a personal computer, a server, a console, a personal digital assistant (PDA), a cell phone, a tablet computing device, a smartphone or any other suitable computing device, or combination of devices. Presenting the above-described functions as being performed by a "system" is not intended to limit the scope of the present application in any way but is intended to provide one example of many embodiments. Indeed, methods, systems and apparatuses disclosed herein may be implemented in localized and distributed forms consistent with computing technology.

It should be noted that some of the system features described in this specification have been presented as modules, in order to more particularly emphasize their implementation independence. For example, a module may be implemented as a hardware circuit comprising custom very large-scale integration (VLSI) circuits or gate arrays, off-the-shelf semiconductors such as logic chips, transistors, or other discrete components. A module may also be implemented in programmable hardware devices such as field

29

programmable gate arrays, programmable array logic, programmable logic devices, graphics processing units, or the like.

A module may also be at least partially implemented in software for execution by various types of processors. An identified unit of executable code may, for instance, comprise one or more physical or logical blocks of computer instructions that may, for instance, be organized as an object, procedure, or function. Nevertheless, the executables of an identified module need not be physically located together but may comprise disparate instructions stored in different locations which, when joined logically together, comprise the module and achieve the stated purpose for the module. Further, modules may be stored on a computer-readable medium, which may be, for instance, a hard disk drive, flash device, random access memory (RAM), tape, or any other such medium used to store data.

Indeed, a module of executable code could be a single instruction, or many instructions, and may even be distributed over several different code segments, among different programs, and across several memory devices. Similarly, operational data may be identified and illustrated herein within modules and may be embodied in any suitable form and organized within any suitable type of data structure. The operational data may be collected as a single data set or may be distributed over different locations including over different storage devices, and may exist, at least partially, merely as electronic signals on a system or network.

It will be readily understood that the components of the application, as generally described and illustrated in the figures herein, may be arranged and designed in a wide variety of different configurations. Thus, the detailed description of the embodiments is not intended to limit the scope of the application as claimed but is merely representative of selected embodiments of the application.

One having ordinary skill in the art will readily understand that the above may be practiced with steps in a different order, and/or with hardware elements in configurations that are different than those which are disclosed. Therefore, although the application has been described based upon these preferred embodiments, it would be apparent to those of skill in the art that certain modifications, variations, and alternative constructions would be apparent.

While preferred embodiments of the present application have been described, it is to be understood that the embodiments described are illustrative only and the scope of the application is to be defined solely by the appended claims when considered with a full range of equivalents and modifications (e.g., protocols, hardware devices, software platforms etc.) thereto.

What is claimed is:

1. An apparatus comprising:

a network interface configured to transmit a user identifier to a plurality of blockchain peers of a blockchain network; and

a processor configured to

receive a plurality of public keys from the plurality of blockchain peers,

receive a plurality of ciphertexts from the plurality of blockchain peers generated based on the user identifier and a corresponding plurality of private keys, generate a combined ciphertext that includes the plurality of ciphertexts and generating a temporary blockchain address of the blockchain network based on the plurality of public keys, and

30

store the combined ciphertext and a transfer of an asset via the temporary blockchain address of the blockchain network.

2. The apparatus of claim 1, wherein the processor is further configured to randomly select the plurality of blockchain peers based on a hash of the user identifier and a pseudo-random algorithm.

3. The apparatus of claim 2, wherein the processor is configured to add together the plurality of public keys to create a single public key and store a blockchain transaction with the transfer of the asset via a blockchain address corresponding to the single public key.

4. The apparatus of claim 2, wherein the processor is configured to hash the user identifier into a string value, slice off a plurality of sub-strings from the string value, and map the plurality of sliced sub-strings to a plurality of respective identifiers of the plurality of blockchain peers in a table.

5. The apparatus of claim 1, wherein the processor is configured to receive a plurality of public keys from a plurality of elliptical curve key pairs generated by the plurality of blockchain peers, where the plurality of ciphertexts are encrypted with a plurality of private keys from the plurality of elliptical curve key pairs.

6. The apparatus of claim 1, wherein the plurality of ciphertexts are added together via a homomorphic process.

7. The apparatus of claim 1, wherein the user identifier comprises an email address of a receiver of the transfer of the asset, and the receiver is not a registered participant of the blockchain network.

8. The apparatus of claim 1, wherein the processor is configured to receive the plurality of private keys from the plurality of blockchain peers, decrypt the ciphertext stored at the temporary blockchain address based on the plurality of private keys to reveal the asset, and transfer the asset to a blockchain address of a recipient.

9. A method comprising:

transmitting a user identifier to a plurality of blockchain peers of a blockchain network;

receiving a plurality of public keys from the plurality of blockchain peers;

receiving a plurality of ciphertexts from the plurality of blockchain peers generated based on the user identifier and a corresponding plurality of private keys;

generating a combined ciphertext that includes the plurality of ciphertexts and generating a temporary blockchain address of the blockchain network based on the plurality of public keys; and

storing the combined ciphertext and a transfer of an asset via the temporary blockchain address of the blockchain network.

10. The method of claim 9, wherein the method further comprises randomly selecting the plurality of blockchain peers based on a hash of the user identifier and a pseudo-random algorithm.

11. The method of claim 10, wherein the generating the temporary blockchain address comprises adding together the plurality of public keys to create a single public key, and storing to blockchain transaction with the transfer of the asset via a blockchain address corresponding to the single public key.

12. The method of claim 10, wherein the randomly selecting comprises hashing the user identifier into a string value, slicing off a plurality of sub-strings from the string value, and mapping the plurality of sliced sub-strings to a plurality of respective identifiers of the plurality of blockchain peers in a table.

31

13. The method of claim 9, wherein the receiving comprises receiving a plurality of public keys from a plurality of elliptical curve key pairs generated by the plurality of blockchain peers, where the plurality of ciphertexts are encrypted with a plurality of private keys from the plurality of elliptical curve key pairs. 5

14. The method of claim 9, wherein the plurality of ciphertexts are added together via a homomorphic process.

15. The method of claim 9, wherein the user identifier comprises an email address of a receiver of the transfer of the asset, and the receiver is not a registered participant of the blockchain network. 10

16. The method of claim 9, wherein the method further comprises receiving the plurality of private keys from the plurality of blockchain peers, decrypting the ciphertext stored at the temporary blockchain address based on the plurality of private keys to reveal the asset, and transferring the asset to a blockchain address of a recipient. 15

17. A computer-readable storage medium comprising instructions, that when read by a processor, cause the processor to perform a method comprising: 20

transmitting a user identifier to a plurality of blockchain peers of a blockchain network;

receiving a plurality of public keys from the plurality of blockchain peers; 25

32

receiving a plurality of ciphertexts from the plurality of blockchain peers generated based on the user identifier and a corresponding plurality of private keys;

generating a combined ciphertext that includes the plurality of ciphertexts and generating a temporary blockchain address of the blockchain network based on the plurality of public keys; and

storing the combined ciphertext and a transfer of an asset via the temporary blockchain address of the blockchain network.

18. The computer-readable storage medium of claim 17, wherein the method further comprises randomly selecting the plurality of blockchain peers based on a hash of the user identifier and a pseudo-random algorithm.

19. The computer-readable storage medium of claim 17, wherein the generating the temporary blockchain address comprises adding together the plurality of public keys to create a single public key, and storing to blockchain transaction with the transfer of the asset via a blockchain address corresponding to the single public key.

20. The computer-readable storage medium of claim 17, wherein the randomly selecting comprises hashing the user identifier into a string value, slicing off a plurality of sub-strings from the string value, and mapping the plurality of sliced sub-strings to a plurality of respective identifiers of the plurality of blockchain peers in a table.

* * * * *